

Les concepts mathématiques sous-jacents





Objectifs de ce module

En suivant ce module vous allez :



- **Apprendre** le passage de l'analogique au numérique et inversement.
- **Découvrir** l'arithmétique dans les bases 2, 8 et 16.
- **Comprendre** comment sont codés les réels selon le format IEEE 754 et comment sont réalisées les opérations dans ce format.
- **Connaître** la manière dont sont codés les caractères dans les ordinateurs grâce aux tables ASCII et UNICODE.
- **Etudier** l'algèbre de Boole qui constitue en partie l'aspect théorique de l'électronique numérique.





Plan du module

Voici les parties que nous allons aborder :



- L'arithmétique binaire.
- Les autres codages.
- L'algèbre de Boole.





Histoire de la numérotation binaire

Les trigrammes de Fu Hsi

- Fu Hsi, considéré comme le premier roi chinois (le premier des trois Augustes), aurait vécu, il y a plus de 5000 ans.
- A l'origine du Yi King (livre des mutations) contenant les **trigrammes** disposés dans l'ordre du « ciel antérieur ».





Histoire de la numérotation binaire

Les trigrammes et les hexagrammes du roi Wen

- Le roi Wen (1150-750 av. J.C), aurait proposé une disposition inversée des trigrammes dit du « ciel postérieur ».
- Il aurait aussi complété le Yi King en introduisant les 64 **hexagrammes** qui sont la combinaison de 2 trigrammes.



Trigrammes supérieur → inférieur ↓	 <i>qián</i> le Ciel	 <i>zhèn</i> le Tonnerre	 <i>kǎn</i> l'Eau	 <i>gèn</i> la Montagne	 <i>kūn</i> la Terre	 <i>xùn</i> le Vent	 <i>lǐ</i> le Feu	 <i>duì</i> la Brume
 <i>qián</i> le Ciel	 1	 34	 5	 26	 11	 09	 14	 43
 <i>zhèn</i> le Tonnerre	 25	 51	 3	 27	 24	 42	 21	 17
 <i>kǎn</i> l'Eau	 6	 40	 29	 4	 7	 59	 64	 47
 <i>gèn</i> la Montagne	 33	 62	 39	 52	 15	 53	 56	 31
 <i>kūn</i> la Terre	 12	 16	 8	 23	 2	 20	 35	 45
 <i>xùn</i> le Vent	 44	 32	 48	 18	 46	 57	 50	 28
 <i>lǐ</i> le Feu	 13	 55	 63	 22	 36	 37	 30	 49
 <i>duì</i> la Brume	 10	 54	 60	 41	 19	 61	 38	 58





Histoire de la numérotation binaire

L'alphabet bilitère de Francis Bacon

- Francis Bacon (1561-1626) a mis au point un **alphabet bilitère** afin de chiffrer des messages.
- Système basé sur les lettres A et B (donc très voisin du codage binaire) dont la combinaison permet de reformer l'alphabet.



	AAA	AAB	ABA	ABB	BAA	BAB	BBA	BBB
AA	A	B	C	D	E	F	G	H
AB	I-J	K	L	M	N	O	P	Q
BA	R	S	T	U-V	W	X	Y	Z





Histoire de la numérotation binaire

L'alphabet bilitère de Francis Bacon

- Placer le message secret dans un « texte de couverture » en utilisant deux écritures légèrement différentes (les polices A et B).
- Pour retrouver le message, traduire le texte en une suite de A et de B en se basant sur le type de caractères puis utiliser la table de correspondance pour trouver les lettres du message.

N	E	P	A	R	T	E	Z	S	U	R	T	O	U	T	P	A	S	S	A	N	S	M	O	I
A	A	B	A	B	B	A	A	B	B	B	A	B	B	A	A	A	B	A	A	B	A	B	B	B
F					U					Y					E					Z				

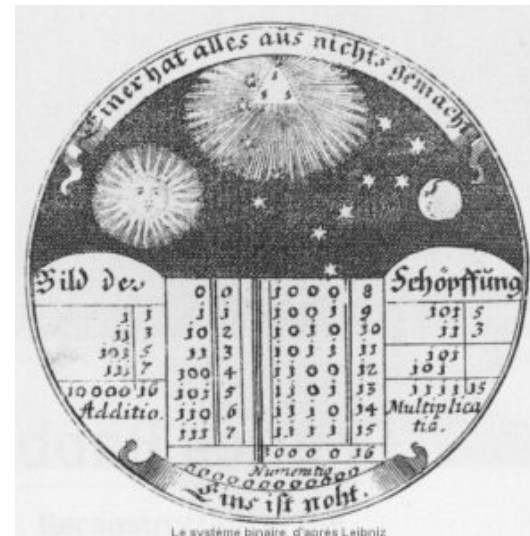




Histoire de la numérotation binaire

L'algèbre binaire de Leibniz

- Gottfried Wilhem Leibniz (1646-1716) a étudié le procédé de chiffrage de Bacon et les trigrammes du Yi King.
- Met au point l'**arithmétique binaire** et expose ses travaux devant l'Académie des Sciences de Paris.
- Publie « Explication de l'arithmétique binaire » en **1703**.



Le système binaire, d'après Leibniz





Histoire de la numérotation binaire

Einer hat alles aus nichts
gemacht.

— Gottfried Wilhem Leibniz





Histoire de la numérotation binaire

Tout est créé à partir de l'unité
(1, Dieu) et du néant (0).

— Gottfried Wilhem Leibniz





Binaire positif vers décimal positif

Méthode de conversion – Le principe

- Un nombre binaire s'écrit avec deux chiffres (1 et 0).
- Il est possible de déterminer la valeur décimale associée grâce à un tableau de correspondance.

Nombre binaire	1	0	1	0	1						
	×	×	×	×	×						
Coefficients	16	8	4	2	1						
	↓	↓	↓	↓	↓						
Nombre décimal	16	+	0	+	4	+	0	+	1	=	21





Binaire positif vers décimal positif

Méthode de conversion – D'un point de vue mathématique

- En base 10, si un nombre N s'écrit xyz_{10} , cela signifie que :
 - $N = x.100 + y.10 + z.1$
 - $N = x.10^2 + y.10^1 + z.10^0$

- Même principe en base 2
- Si un nombre M s'écrit $abcde_2$, cela signifie que :
 - $M = a.2^4 + b.2^3 + c.2^2 + d.2^1 + e.2^0$
 - $M = a.16 + b.8 + c.4 + d.2 + e.1$





Binaire positif vers décimal positif

Exemple

■ Si $N = 1011010_2$, nous obtenons :

$$\blacksquare N = 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$\blacksquare N = 1 \times 64 + 0 \times 32 + 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1$$

$$\blacksquare N = 90_{10}$$





Binaire positif vers décimal positif

Exercice – Traduire les nombres binaires en base 10

■ $N = 101010_2 :$

■ $N = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

■ $N = 1 \times 32 + 0 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1$

■ $N = 42_{10}$

■ $M = 1001110_2 :$

■ $M = 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

■ $M = 1 \times 64 + 0 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1$

■ $M = 78_{10}$





Décimal positif vers binaire positif

Première méthode (les divisions successives)

- Soit un nombre $N = xyz_{10}$ écrit en base 10, nous devons écrire ce même nombre sous la forme :

$$N = N_0 = a_n \cdot 2^n + a_{n-1} \cdot 2^{n-1} \dots + a_0 \cdot 2^0$$

- Pour cela, on effectue une série de divisions par 2 :
 - Si on divise N_0 par 2, on obtient :
 - un reste qui correspond à a_0 ;
 - un quotient qui devient le nombre N_1 qui sera divisé par 2 au coup suivant ...
 - On s'arrête quand N_i est égal à 0 ou à 1.





Décimal positif vers binaire positif

Exemple

■ Si $N = 90_{10}$, nous obtenons :

$$\begin{array}{r|l} 90 & 2 \\ \hline -90 & 45 \\ \hline 0 & \end{array} \quad \begin{array}{r|l} 45 & 2 \\ \hline -44 & 22 \\ \hline 1 & \end{array} \quad \begin{array}{r|l} 22 & 2 \\ \hline -22 & 11 \\ \hline 0 & \end{array} \quad \begin{array}{r|l} 11 & 2 \\ \hline -10 & 5 \\ \hline 1 & \end{array} \quad \begin{array}{r|l} 5 & 2 \\ \hline -4 & 2 \\ \hline 1 & \end{array} \quad \begin{array}{r|l} 2 & 2 \\ \hline -2 & 1 \\ \hline 0 & \end{array} \quad \begin{array}{r|l} 1 & 2 \\ \hline -0 & 0 \\ \hline 1 & \end{array}$$

■ En lisant de droite à gauche, on retrouve **1011010_2** .





Décimal positif vers binaire positif

Exercice – Traduire le nombre décimal en base 2

■ $N = 73_{10}$:

$\begin{array}{r l} 73 & 2 \\ \hline -72 & 36 \\ \hline 1 & \end{array}$	$\begin{array}{r l} 36 & 2 \\ \hline -36 & 18 \\ \hline 0 & \end{array}$	$\begin{array}{r l} 18 & 2 \\ \hline -18 & 9 \\ \hline 0 & \end{array}$	$\begin{array}{r l} 9 & 2 \\ \hline -8 & 4 \\ \hline 1 & \end{array}$	$\begin{array}{r l} 4 & 2 \\ \hline -4 & 2 \\ \hline 0 & \end{array}$	$\begin{array}{r l} 2 & 2 \\ \hline -2 & 1 \\ \hline 0 & \end{array}$	$\begin{array}{r l} 1 & 2 \\ \hline -0 & 0 \\ \hline 1 & \end{array}$
--	--	---	---	---	---	---

■ En lisant de droite à gauche, on trouve **1001001_2** .





Décimal positif vers binaire positif

Exercice – Traduire le nombre décimal en base 2

■ $N = 57_{10}$:

$\begin{array}{r l} 57 & 2 \\ \hline -56 & 28 \\ \hline 1 & \end{array}$	$\begin{array}{r l} 28 & 2 \\ \hline -28 & 14 \\ \hline 0 & \end{array}$	$\begin{array}{r l} 14 & 2 \\ \hline -14 & 7 \\ \hline 0 & \end{array}$	$\begin{array}{r l} 7 & 2 \\ \hline -6 & 3 \\ \hline 1 & \end{array}$	$\begin{array}{r l} 3 & 2 \\ \hline -2 & 1 \\ \hline 1 & \end{array}$	$\begin{array}{r l} 1 & 2 \\ \hline -0 & 0 \\ \hline 1 & \end{array}$
--	--	---	---	---	---

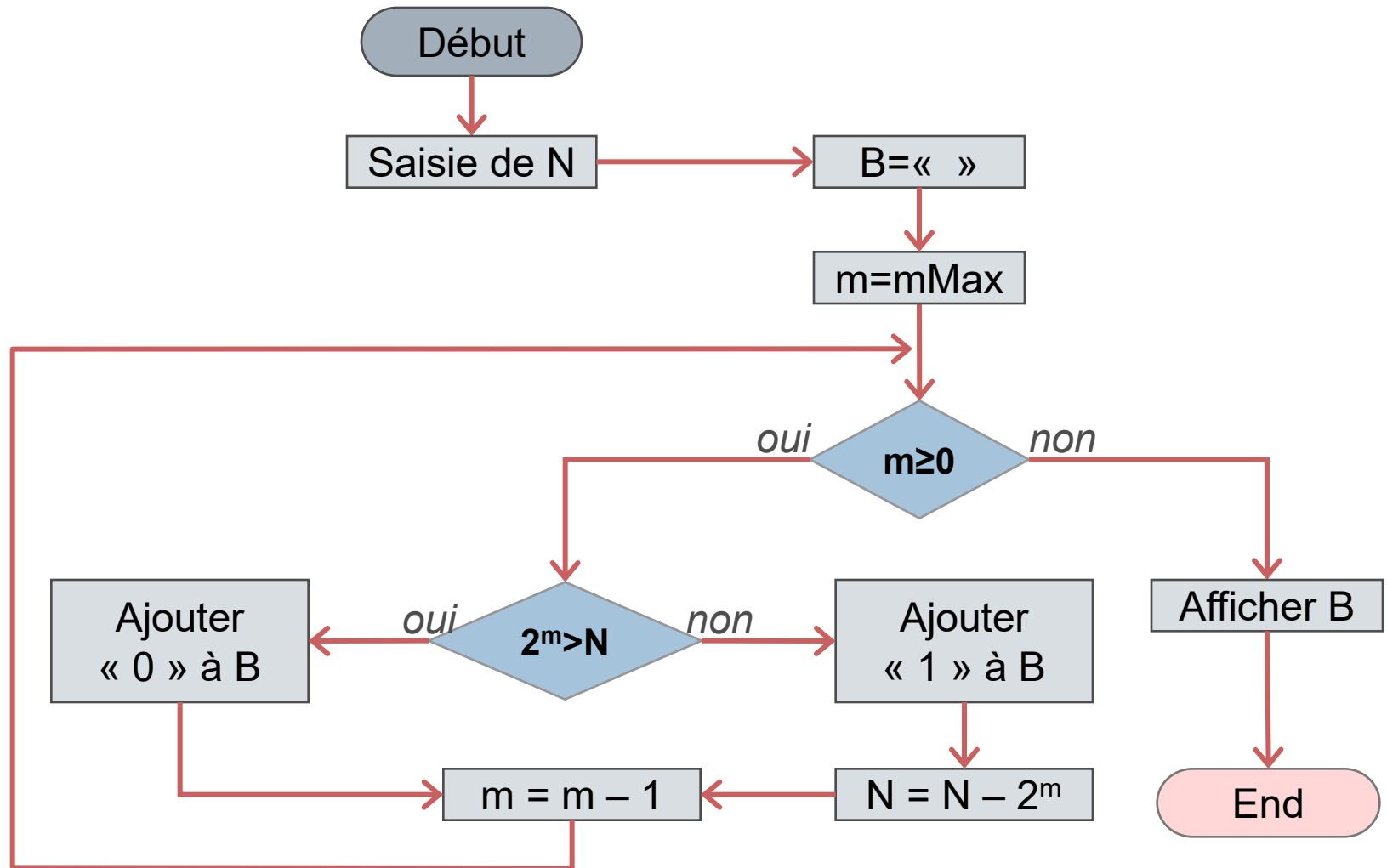
■ En lisant de droite à gauche, on trouve 111001_2





Décimal positif vers binaire positif

Seconde méthode (algorithme d'écriture)





Décimal positif vers binaire positif

Seconde méthode (algorithme d'écriture)

- On pose les variables suivantes :
 - **N** est le nombre décimal à transformer.
 - **B** est le nombre binaire obtenu à l'issu du traitement.
 - **M** est la puissance de 2 « en cours »
 - **mMax** est la puissance de 2 de départ, cette variable entière correspond également au nombre de chiffres avec lesquels sera écrit B. Généralement ce nombre est fixé de la manière suivante :
 - on prend la puissance de 2 (2^k), immédiatement inférieur à N ;
 - on fixe alors $mMax = k$.





Décimal positif vers binaire positif

Exemple

■ Si $N = 26_{10}$ et $mMax = 5_{10}$, nous obtenons :

M	Test	N	B
départ		26	« »
5	$2^5=32 > 26$	26	« 0 »
4	$2^4=16 \leq 26$	$10 = 26 - 16$	« 01 »
3	$2^3=8 \leq 10$	$2 = 10 - 8$	« 011 »
2	$2^2=4 > 2$	2	« 0110 »
1	$2^1=2 \leq 2$	$0 = 2 - 2$	« 01101 »
0	$2^0=1 > 0$	0	« 011010 »





Décimal positif vers binaire positif

Exercice – Traduire le nombre décimal en base 2

■ $N = 196_{10}$ avec $mMax=7_{10}$

M	Test	N	B
départ		196	« »
7	$2^7=128 \leq 196$	$68 = 196 - 128$	« 1 »
6	$2^6=64 \leq 68$	$4 = 68 - 64$	« 11 »
5	$2^5=32 > 4$	4	« 110 »
4	$2^4=16 > 4$	4	« 1100 »
3	$2^3=8 > 4$	4	« 11000 »
2	$2^2=4 \leq 4$	$0 = 4 - 4$	« 110001 »
1	$2^1=2 > 0$	0	« 1100010 »
0	$2^0=1 > 0$	0	« 11000100 »





Décimal positif vers binaire positif

Exercice – Traduire le nombre décimal en base 2

■ $N = 201_{10}$ avec $mMax = 7_{10}$

M	Test	N	B
départ		201	« »
7	$2^7 = 128 \leq 201$	$73 = 201 - 128$	« 1 »
6	$2^6 = 64 \leq 73$	$9 = 73 - 64$	« 11 »
5	$2^5 = 32 > 9$	9	« 110 »
4	$2^4 = 16 > 9$	9	« 1100 »
3	$2^3 = 8 \leq 9$	$1 = 9 - 8$	« 11001 »
2	$2^2 = 4 > 1$	1	« 110010 »
1	$2^1 = 2 > 1$	1	« 1100100 »
0	$2^0 = 1 \leq 1$	$0 = 1 - 1$	« 11001001 »





Addition des binaires positifs

Méthode manuelle avec propagation de la retenue

- Même principe que l'addition « à la main » en base 10.
- Retenue créée lorsque la somme dépasse 1_2 .
- 4 cas de figure :
 - $0_2 + 0_2 = 0_2$
 - $0_2 + 1_2 = 1_2$
 - $1_2 + 0_2 = 1_2$
 - $1_2 + 1_2 = 1 \times 2^1 + 0 \times 2^0 = 10_2$ (on a créé une retenue)





Addition des binaires positifs

Exemple

- Additionner 10111011_2 et 1101_2

Addition binaire	Addition décimale
1 1 1 1 1 1 1 0 1 1 1 0 1 1 + 1 1 0 1 1 1 0 0 1 0 0 0	1 1 1 8 7 + 1 3 2 0 0





Addition des binaires positifs

Exercice

- Additionner 1111000_2 et 10000111_2

Addition binaire	Addition décimale
$\begin{array}{r} 11110000 \\ + 10000111 \\ \hline 10111011 \end{array}$	$\begin{array}{r} 240 \\ + 135 \\ \hline 375 \end{array}$





Addition des binaires positifs

Exercice

- Additionner 11100_2 et 10111111_2

Addition binaire	Addition décimale
1111 11100 + 10111111 11011011	1 28 + 191 219





Soustraction des binaires positifs

Méthode manuelle avec propagation de la retenue négative

- Même principe que la soustraction « à la main » en base 10.
- Retenue créée lorsque la différence est inférieure à 0_2
- 4 cas de figure :
 - $0_2 - 0_2 = 0_2$
 - $1_2 - 1_2 = 0_2$
 - $1_2 - 0_2 = 1_2$
 - $0_2 - 1_2 = 10_2 - 1_2 - 10_2 = 1_2 - 10_2$ (retenue négative)





Soustraction des binaires positifs

Exercice

- Soustraire 101101_2 à 10010011_2

Soustraction binaire	Soustraction décimale
$\begin{array}{r} \\ \\ 1 0 \\ - \\ \hline 1 0 \end{array}$	$\begin{array}{r} \\ \\ 1 3 \\ - \\ \hline \end{array}$





Soustraction des binaires positifs

Exercice

- Soustraire 1100111_2 à 10110101_2

Soustraction binaire	Soustraction décimale
$\begin{array}{r} \overset{-1}{1} \overset{-1}{1} \overset{-1}{0} \overset{-1}{1} \\ - \\ \hline 0 1 0 1 1 1 \end{array}$	$\begin{array}{r} \overset{-1}{1} \\ - \\ \hline 7 8 \end{array}$





Codage des nombres binaires négatifs

Le principe

- Nombre signé.
- 3 codages possibles :
 - coder par bit de signe et valeur absolue ;
 - utiliser le complément à 1 ;
 - utiliser le complément à 2.





Codage des nombres binaires négatifs

Codage par bit de signe et valeur absolue

- Signe porté par le bit de poids fort :
 - 0 pour une valeur positive ;
 - 1 pour une valeur négative.
- Les autres bits servent à coder la valeur absolue.

Valeur positive	Valeur négative
$00001011_2 = 11_{10}$	$10001011_2 = -11_{10}$
$00000000_2 = 0_{10}$	$10000000_2 = -0_{10}$





Codage des nombres binaires négatifs

Complément à 1

- Signe encore porté par le bit de poids fort.
- On passe d'une valeur à son opposée en inversant les bits.

Valeur positive	Valeur négative
$00001011_2 = 11_{10}$	$11110100_2 = -11_{10}$
$00000000_2 = 0_{10}$	$11111111_2 = -0_{10}$





Codage des nombres binaires négatifs

Complément à 2

- Signe toujours porté par le bit de poids fort.
- On construit le complément à 2 en ajoutant 1 au complément à 1 décrit précédemment.
- S'effectue en ayant **fixé le nombre de bits utilisés** pour coder les nombres (ici 8 bits).

Valeur positive	Valeur négative
$00001011_2 = 11_{10}$	$11110100_2 + 1_2 = 11110101_2 = -11_{10}$
$00000000_2 = 0_{10}$	$11111111_2 + 1_2 = 00000000_2 = -0_{10}$





Codage des nombres binaires négatifs

Complément à 2

- Encore appelé « complément vrai ».
- On somme un nombre et son contraire : on obtient bien 0 car **on perd la dernière retenue** (addition sur un nombre limité de bits).

Addition binaire	Addition décimale
$\begin{array}{r} 1\ 1\ 1\ 1\ 1\ 1\ 1 \\ 1\ 1\ 1\ 1\ 0\ 1\ 0\ 1 \\ +\ 0\ 0\ 0\ 0\ 1\ 0\ 1\ 1 \\ \hline 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0 \end{array}$	$\begin{array}{r} -1\ 1 \\ +\ 1\ 1 \\ \hline 0\ 0 \end{array}$
	
$0\ 0\ 0\ 0\ 0\ 0\ 0\ 0$	





Soustraction des binaires positifs (bis)

Méthode par addition du complément à 2

- Plus facilement transposable en électronique.
- Nécessite de fixer le nombre de bits sur lequel on travaille.
- Supposons la soustraction $A - B$, le processus est le suivant :
 - construire le complément à 2 de B (on obtient $-B$)
 - sommer A et $-B$





Soustraction des binaires positifs (bis)

Exemple

■ Soustraire 16_{10} à 47_{10}

■ Etape 1 : Traduire en binaire

Décimal	Binaire	Complément à 1	Complément à 2
47	00101111	11010000	11010001
16	00010000	11101111	11110000

■ Etape 2 : Effectuer l'addition

Addition binaire	Soustraction décimale
1 1 0 0 1 0 1 1 1 1 + 1 1 1 1 0 0 0 0 1 0 0 0 1 1 1 1	4 7 - 1 6 3 1





Soustraction des binaires positifs (bis)

Exercice

■ Soustraire 95_{10} à 123_{10}

■ Etape 1 : Traduire en binaire

Décimal	Binaire	Complément à 1	Complément à 2
123	01111011	10000100	10000101
95	01011111	10100000	10100001

■ Etape 2 : Effectuer l'addition

Addition binaire	Soustraction décimale
1 1 1 1 1 0 1 1 0 1 1 1 1 0 1 1 + 1 0 1 0 0 0 0 1 1 0 0 0 1 1 1 0 0	-1 1 2 13 - 9 5 2 8





Soustraction des binaires positifs (bis)

Exercice

- Soustraire 162_{10} à 213_{10}
 - Etape 1 : Traduire en binaire

Décimal	Binaire	Complément à 1	Complément à 2
213	11010101	00101010	00101011
162	10100010	01011101	01011110

- Etape 2 : Effectuer l'addition

Addition binaire	Soustraction décimale
$\begin{array}{r} 1 \quad 1 \quad 1 \quad 1 \\ 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \\ + 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \\ \hline 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \end{array}$	$\begin{array}{r} -1 \\ 2 \quad 11 \quad 3 \\ - 1 \quad 6 \quad 2 \\ \hline 0 \quad 5 \quad 1 \end{array}$





Multiplication des binaires

La multiplication par une puissance de 2

- Se traduit par un **décalage vers la gauche** du nombre multiplié.
- Nombre de crans correspondant à la puissance de deux de l'élément multiplicateur.
- Supposons deux nombre A et B, tel que $B=2^N$:
 - B s'écrit « 1 » suivi de N « 0 »
 - $A \times B$ s'écrit donc A suivi de N « 0 »





Multiplication des binaires

Exemple de multiplication par une puissance de 2

- Soient $A = 101101110111001_2$ et $B = 10000_2$.
- On obtient $A \times B = 1011011101110010000_2$.
- Si nous traduisons ces 3 nombres en base 10, nous avons :
 - $A = 23481_{10}$
 - $B = 16_{10}$
 - $A \times B = 375696_{10} = 23481_{10} \times 16_{10}$





Multiplication des binaires

Le principe de la multiplication de deux binaires

- Plusieurs multiplications successives par des puissances de 2 croissantes (plusieurs décalages).
- Une addition des produits intermédiaires pour former le résultat final.





Multiplication des binaires

Exemple

- Multiplier 10100111_2 à 1100_2

Multiplication binaire

$$\begin{array}{r} \\ \\ \\ \\ \\ \\ \\ + \\ \hline 1 1 1 1 1 1 0 \end{array}$$

Multiplication décimale

$$\begin{array}{r} \\ \\ \\ + \\ \hline 2 \end{array}$$





Multiplication des binaires

Exemple

- Multiplier 101001_2 à 101001_2

Multiplication binaire	Multiplication décimale
1 1 1 1 101001 101001 101001 000000 000000 001001 000000 101001 + 101001 1101001001	41 41 41 1640 + 1640 1681





Division des binaires

La division par une puissance de 2

- Se traduit par un **décalage vers la droite** du nombre divisé.
- Nombre de crans correspondant à la puissance de deux de l'élément diviseur.
- Supposons deux nombre A et B, tel que $B=2^N$:
 - B s'écrit « 1 » suivi de N « 0 »
 - A / B s'écrit donc « A privé des N bits de poids faible »
- La **division entière** se traduit donc par la **perte des bits de poids faible**.





Division des binaires

Exemple de division par une puissance de 2

■ Soient $A = 101101110111001_2$ et $B = 10000_2$.

■ On obtient $A / B = 10110111011\textcolor{red}{1001}_2$.

■ Si nous traduisons ces 3 nombres en base 10, nous avons :

■ $A = 23481_{10}$

■ $B = 16_{10}$

■ $A / B = 1467_{10}$





Division des binaires

Le principe de la division de deux binaires

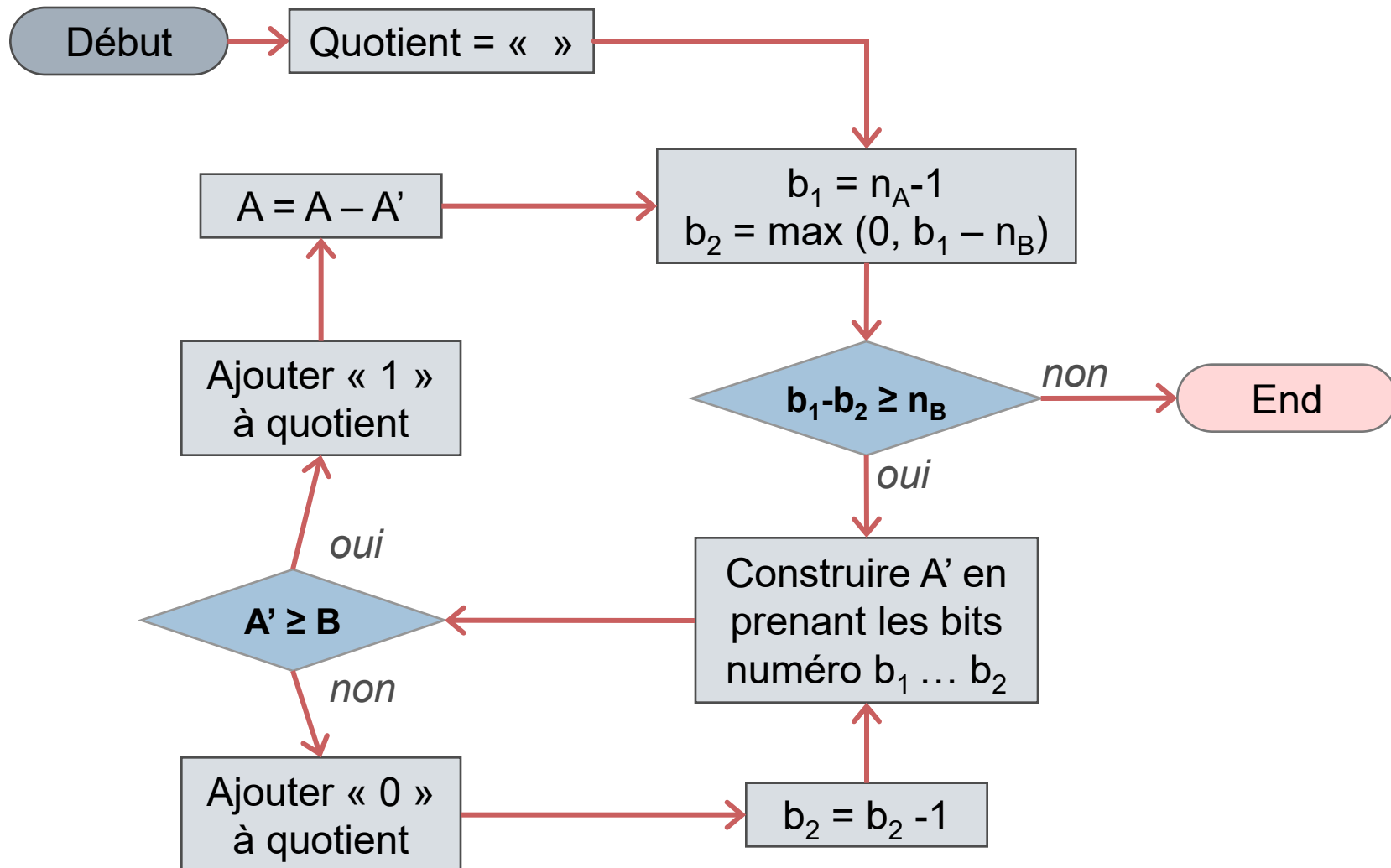
- La division manuelle de deux binaires s'effectuent de la même manière que la division manuelle de deux décimaux.
- Pour décrire le processus, nous supposons deux nombres binaires A et B. Nous posons alors les notations suivantes :
 - n_A est la taille (en nombre de chiffres) du nombre A
 - n_B est la taille (en nombre de chiffres) du nombre B
 - b_1 et b_2 sont des numéros de bits (pour un binaire ayant une taille de N, les bits sont numérotés de 0 à N-1, le bit 0 étant celui de poids faible)
 - A' est un nombre binaire formée en prenant les bits de A dont les numéros sont compris entre b_1 et b_2





Division des binaires

Le principe de la division de deux binaires (A/B)





Division des binaires

Exemple

- Diviser 10110111_2 par 1100_2

Division binaire	Division décimale
$\begin{array}{r} 10110111 \\ - 1100 \\ \hline 1010111 \\ - 1100 \\ \hline 100111 \\ - 1100 \\ \hline 1111 \\ - 1100 \\ \hline 11 \end{array}$	$\begin{array}{r} 183 \\ - 12 \\ \hline 63 \\ - 60 \\ \hline 3 \end{array}$
$\begin{array}{r} 1100 \\ \hline 01111 \end{array}$	$\begin{array}{r} 12 \\ \hline 15 \end{array}$

- On a bien $1111_2 = 15_{10}$ pour le quotient et $11_2 = 3_{10}$ pour le reste de la division.





Division des binaires

Exercice

- Diviser 1000110111_2 par 1011_2

Division binaire	Division décimale
1000110111 - 1011 ----- 11010111 - 1011 ----- 100111 - 1011 ----- 10001 - 1011 ----- 110	567 - 55 ----- 17 - 11 ----- 6

- On trouve un quotient égal à $51_{10} = 110011_2$ et un reste égal à $6_{10} = 110_2$.





Division des binaires

Exercice

- Diviser 1001001100_2 par 1110_2

Division binaire	Division décimale
1001001100 - 1110 ----- 10001100 - 1110 ----- 11100 - 1110 ----- 00	588 - 56 ----- 28 - 28 ----- 0

- On trouve un quotient égal à $42_{10} = 101010_2$ et pas de reste.





Pause-réflexion sur cette 2^{ème} partie

Avez-vous des questions ?





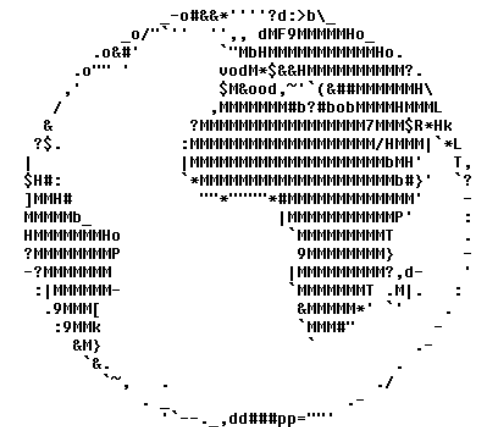
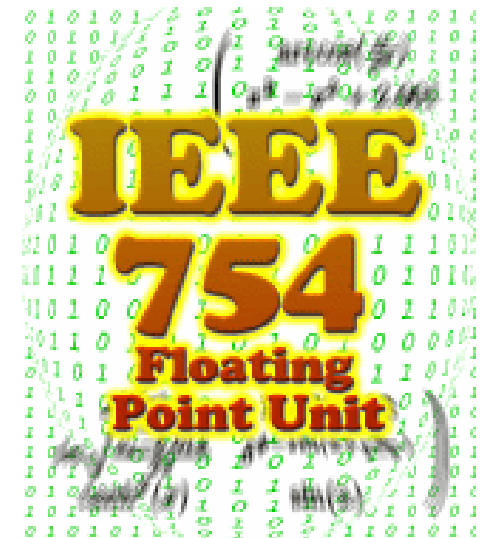
Les autres codage



Plan de la partie

Voici les chapitres que nous allons aborder :

- Les binaires à virgule fixe.
- Les binaires à virgule flottante.
- IEEE 754 – Configurations particulières.
- IEEE 754 - Addition et soustraction.
- IEEE 754 - Multiplication et division.
- Le codage hexadécimal.
- Le codage octal et le DCB.
- Le codage des caractères.





Les binaires à virgule fixe

La première façon de représenter les nombres réels

- Les explications précédentes restent valables.
- Un nombre de bits **fixés** portent des puissances de deux **négligables**.
- Si un nombre M s'écrit $abc.de_2$, cela signifie que :
 - $M = a \times 2^2 + b \times 2^1 + c \times 2^0 + d \times 2^{-1} + e \times 2^{-2}$
 - $M = a \times 4 + b \times 2 + c \times 1 + d \times 0,5 + e \times 0,25$





Les binaires à virgule fixe

Exemple

■ Si $N = 01011.011_2$, nous obtenons :

■ $N = 2^3 + 2^1 + 2^0 + 2^{-2} + 2^{-3}$

■ $N = 8 + 2 + 1 + 1/4 + 1/8$

■ $N = (64 + 16 + 8 + 2 + 1) / 8$

■ $N = 91/8$

■ $N = 11.375_{10}$





Les binaires à virgule fixe

Exercice – Traduire les nombres binaires en base 10

■ $N = 111.010_2 :$

■ $N = 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3}$

■ $N = 1 \times 4 + 1 \times 2 + 1 \times 1 + 0 \times 0.5 + 1 \times 0.25 + 0 \times 0.125$

■ $N = 7.25_{10}$

■ $M = 101.1010_2 :$

■ $M = 2^2 + 2^0 + 2^{-1} + 2^{-3}$

■ $M = 4 + 1 + 0.5 + 0.125$

■ $M = 5.625_{10}$





Les binaires à virgule fixe

Les limites de la méthode de codage

- Ne permet pas de gérer des grands nombres ou des petits nombres simultanément.
- Provoque des **erreurs d'arrondis**.





Les binaires à virgule fixe

Les limites de la méthode de codage – Exemple

- Considérons deux binaires non signés, codés sur 8 bits avec deux bits pour la partie décimale :

$$11_{10} = 001011.00_2 \text{ et } 8_{10} = 001000.00_2$$

- On fait $11/8$:
 - On aurait dû obtenir $1.375_{10} = 000001.011_2$
 - On obtient $1.25_{10} = 000001.01_{10}$
 - On a supprimé le dernier bit.



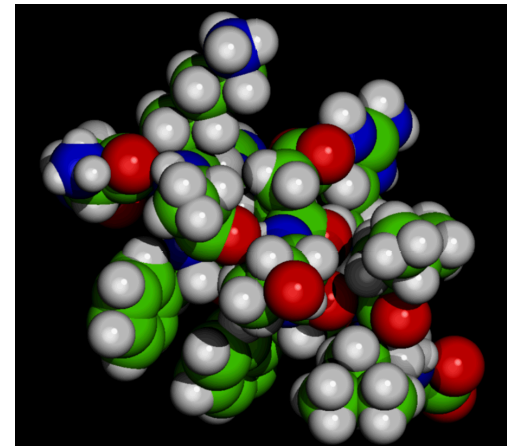
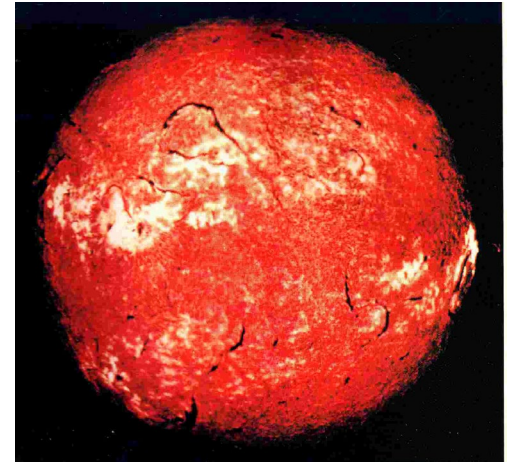


Les binaires à virgule flottante

La notation scientifique en base 10

- Permet de manipuler des grands nombres et/ou des petits nombres simultanément.
- Séparation de la précision (**mantisse**) de l'ordre de grandeur (**exposant**).
- De la forme :

$$N = \text{mantisse} \times 10^{\text{exposant}}$$





Les binaires à virgule flottante

La norme IEEE 754

- La norme **IEEE 754** publiée en 1985.
- 3 modes de représentation des nombres binaires à virgules flottantes :
 - les nombres « simple précision » codés sur 32 bits
 - les nombres « double précision » codés sur 64 bits
 - les nombres « précision étendue » codés sur 80 bits





Les binaires à virgule flottante

La norme IEEE 754 – Les 3 champs

- Les nombres sont codés avec 3 champs :
 - un bit de signe (0 positif et 1 négatif) ;
 - un champ exposant ;
 - un champ mantisse.

	Signe	Exposant	Mantisse
Simple précision	1 bit	8 bits	23 bits
Double précision	1 bit	11 bits	52 bits
Précision étendue	1 bit	15 bits	64 bits

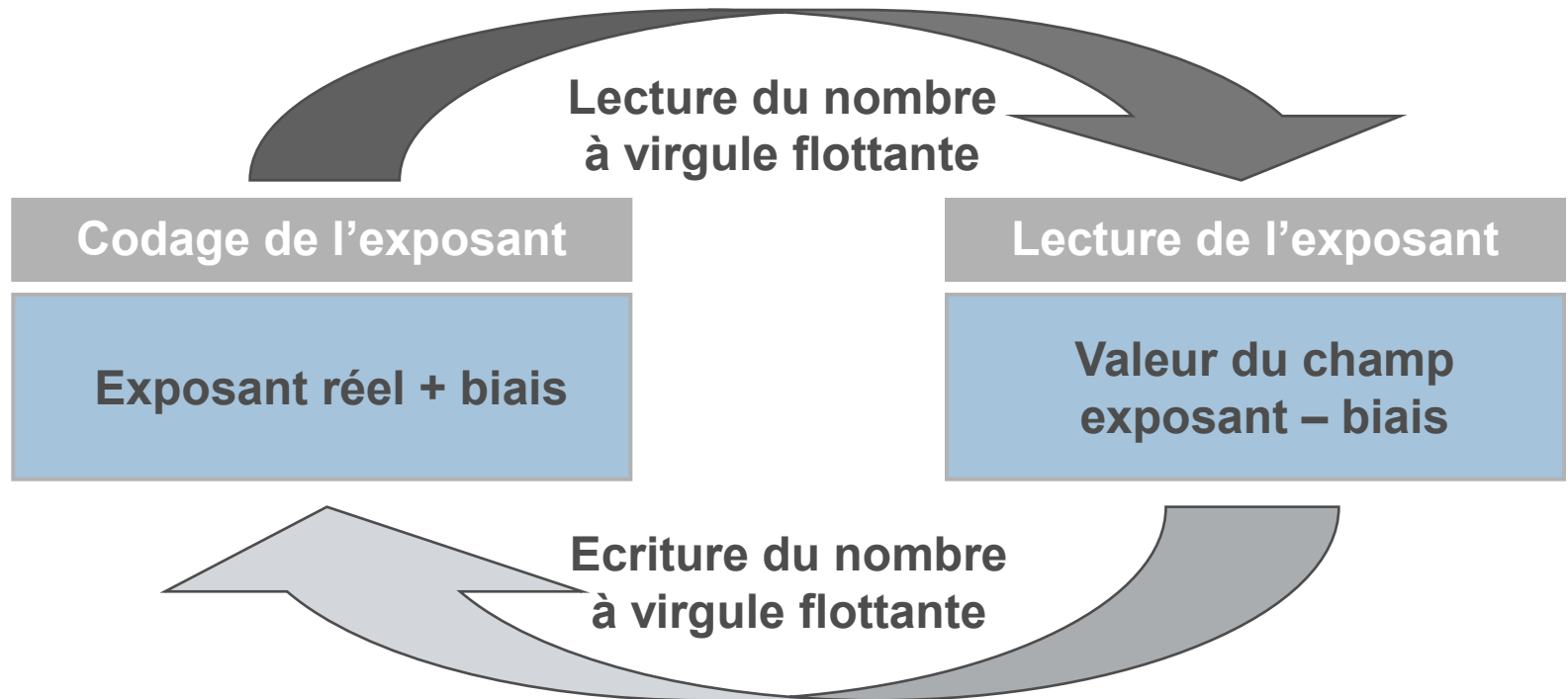




Les binaires à virgule flottante

La norme IEEE 754 – Le champ exposant

- Exposants positifs et exposants négatifs.
- Utilisation d'un biais (qui permet d'avoir un champ exposant toujours positif).





Les binaires à virgule flottante

La norme IEEE 754 – Le champ exposant – Exemples

■ Codage de l'exposant :

	Exposant réel	Biais	Valeur codée
Simple précision	0	127	127
Double précision	-27	1023	996
Précision étendue	-500	16383	15883

■ Lecture de l'exposant

	Valeur codée	Biais	Exposant réel
Simple précision	127	127	0
Double précision	996	1023	-27
Précision étendue	15883	16383	-500





Les binaires à virgule flottante

La norme IEEE 754 – Le champ mantisse

- La mantisse est **normalisée**
- La **partie entière** ne contient qu'**un chiffre différent de 0**.
- Assure l'**unicité de l'écriture** d'un nombre par la notation scientifique.

$$N = abcde0000 \times 10^{-4}$$

$$N = abc.de \times 10^2$$

$$N = abcde000 \times 10^{-3}$$

$$N = ab.cde \times 10^3$$

$$N = abcde00 \times 10^{-2}$$

$$N = a.bcde \times 10^4$$

$$N = abcde0 \times 10^{-1}$$

$$N = 0.abcde \times 10^5$$

$$N = abcde \times 10^0$$

$$N = 0.0abcde \times 10^6$$

$$N = abcd.e \times 10^1$$

$$N = 0.00abcde \times 10^7$$





Les binaires à virgule flottante

La norme IEEE 754 – Le champ mantisse – Le bit caché

- En base 2, le chiffre différent de 0 est 1.
- Pas besoin de représenter la partie entière (bit caché).
- On gagne 1 bit pour la fraction (plus de précision).





Les binaires à virgule flottante

Conversion binaire IEEE 754 vers décimal réel

- On suppose un nombre binaire N, codé selon le format IEEE 754

signe

exposant

mantisse

- Le nombre N converti en base 10 est calculé par la formule suivante :

$$N = (-1)^{\text{signe}} \times \left(1 + \sum_{i=1}^{\text{taille}(\text{mantisse})} \text{mantisse}_i \times 2^{-i} \right) \times 2^{\text{exp}-\text{biais}}$$





Les binaires à virgule flottante

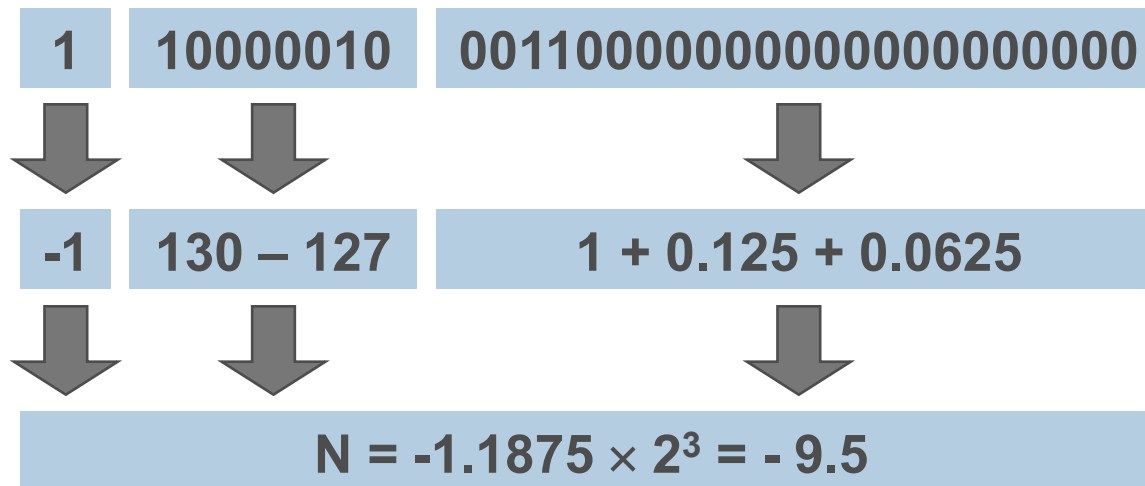
Conversion binaire IEEE 754 vers décimal réel – Exemple

■ Soit $N = 1\ 10000010\ 001100000000000000000000_2$

■ N est codé en simple précision donc

■ $\text{taille(mantisse)} = 23$

■ $\text{biais} = 127$





Les binaires à virgule flottante

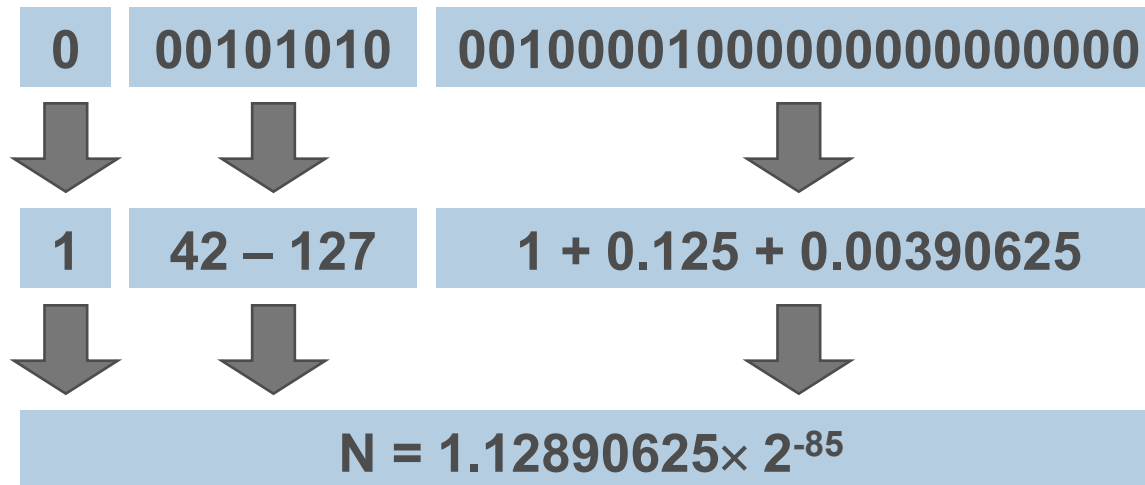
Conversion binaire IEEE 754 vers décimal réel – Exercice

■ Soit $N = 0\ 00101010\ 001000010000000000000000_2$

■ N est codé en simple précision donc

■ $\text{taille}(\text{mantisse}) = 23$

■ $\text{biais} = 127$





Les binaires à virgule flottante

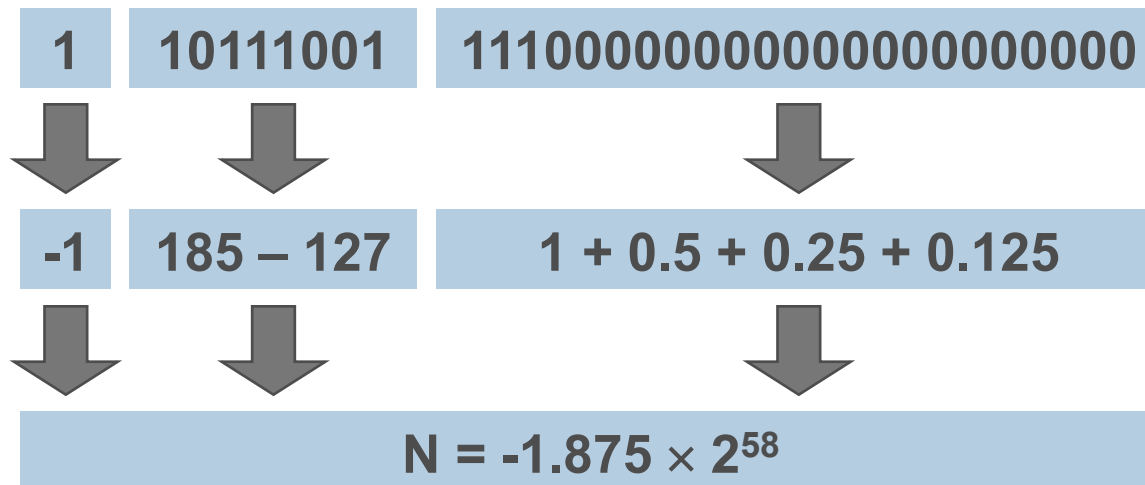
Conversion binaire IEEE 754 vers décimal réel – Exercice

■ Soit $N = 1\ 10111001\ 111000000000000000000000_2$

■ N est codé en simple précision donc

■ $\text{taille}(\text{mantisse}) = 23$

■ $\text{biais} = 127$





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754

- La conversion s'effectue en trois étapes :
 - conversion du nombre réel écrit en base 10, vers la base 2 en utilisant une version modifiée de **l'algorithme d'écriture** (le test d'arrêt est **$N > 0$** et non $M \geq 0$) ;
 - **normalisation** du nombre binaire obtenu ;
 - troncature éventuelle de la mantisse selon la précision choisie.

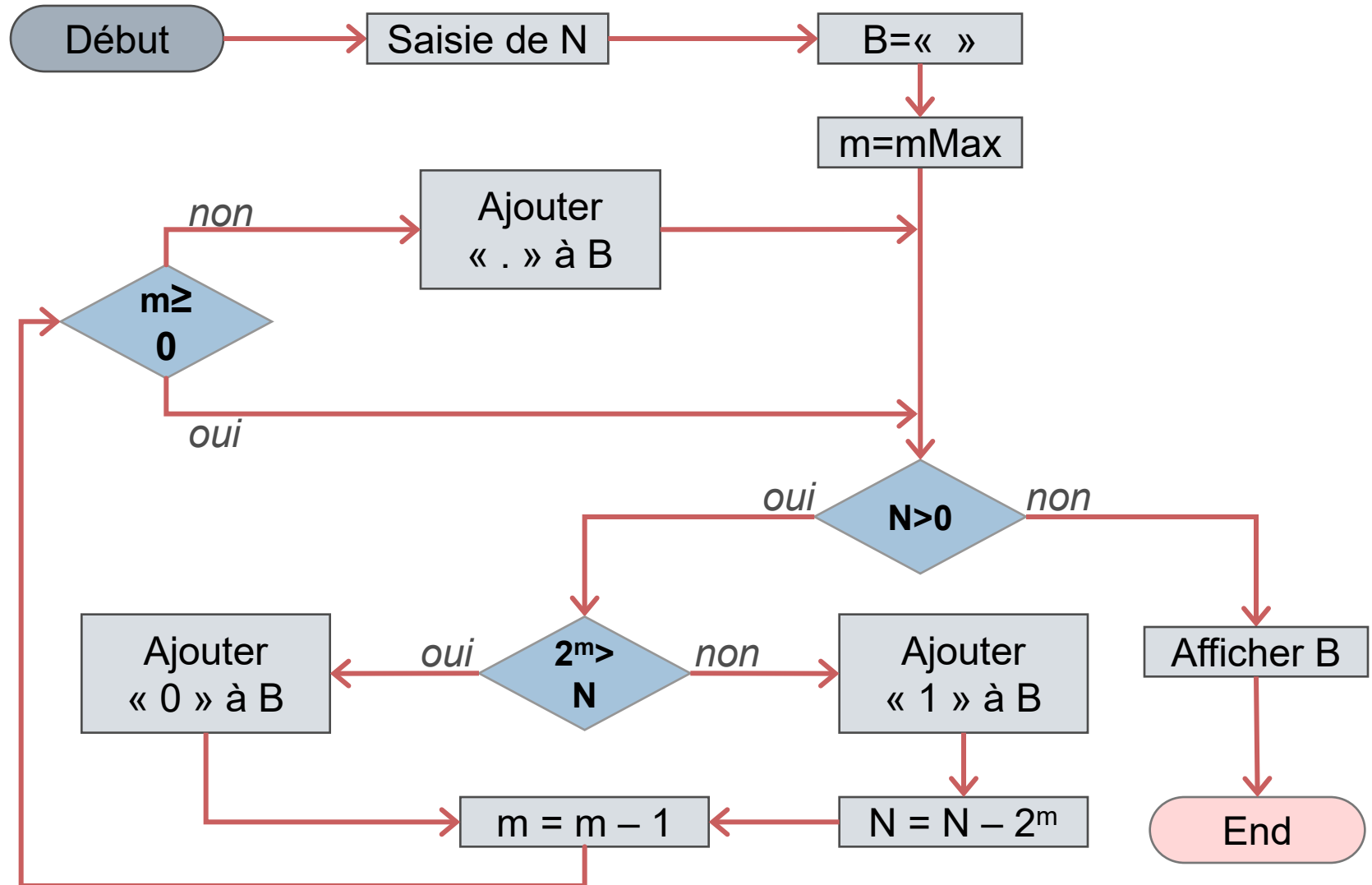
- Peut conduire à des valeurs approchées qui entraînent de **erreurs d'arrondis**.





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

■ $N = 54.265625_{10}$ avec $mMax=7_2$

M	Test	N	B
départ		54.265625	« »
7	$2^7=128 > 54.265625$	54.265625	« 0 »
6	$2^6=64 > 54.265625$	54.265625	« 00 »
5	$2^5=32 \leq 54.265625$	$22.265625=54.265625-32$	« 001 »
4	$2^4=16 \leq 22.265625$	$6.265625=22.265625-16$	« 0011 »
3	$2^3=8 > 6.265625$	6.265625	« 00110 »
2	$2^2=4 \leq 6.265625$	$2.265625=6.265625-4$	« 001101 »
1	$2^1=2 \leq 2.265625$	$0.265625=2.265625-2$	« 0011011 »
0	$2^0=1 > 0.265625$	0.265625	« 00110110. »
-1	$2^{-1}=0.5 > 0.265625$	0.265625	« 00110110.0 »
-2	$2^{-2}=0.25 \leq 0.265625$	$0.015625=0.265625-0.25$	« 00110110.01 »
-3	$2^{-3}=0.125 > 0.015625$	0.015625	« 00110110.010 »
-4	$2^{-4}=0.0625 > 0.015625$	0.015625	« 00110110.0100 »
-5	$2^{-5}=0.03125 > 0.015625$	0.015625	« 00110110.01000 »
-6	$2^{-6}=0.015625 \leq 0.015625$	$0=0.015625-0.015625$	« 00110110.010001 »





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

- $N = 54.265625_{10}$ est donc égal à 110110.010001_2
- Nous devons maintenant normaliser ce nombre, nous obtenons donc $N = 1.10110010001_2 \times 2^5_{10}$
- Finalement, le codage de N selon le format IEEE 754 est :
 - Bit de signe à 0 (le nombre est positif)
 - Champ exposant = $5_{10} + 127_{10}$ soit 10000100_2
 - Champ mantisse = $101100100010000000000000_2$
- On obtient donc **$0\ 10000100\ 101100100010000000000000_2$**





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

■ $N = 11.99609375_{10}$ avec $mMax=3_{10}$

M	Test	N	B
départ		11.99609375	« »
3	$2^3=8 \leq 11.99609375$	$3.99609375=11.99609375-8$	« 1 »
2	$2^2=4 > 3.99609375$	3.99609375	« 10 »
1	$2^1=2 \leq 3.99609375$	$1.99609375=3.99609375-2$	« 101 »
0	$2^0=1 \leq 1.99609375$	$0.99609375=1.99609375-1$	« 1011. »
-1	$2^{-1}=0.5 \leq 0.99609375$	$0.49609375=0.99609375-0.5$	« 1011. 1 »
-2	$2^{-2}=0.25 \leq 0.49609375$	$0.24609375=0.49609375-0.25$	« 1011. 11 »
-3	$2^{-3}=0.125 \leq 0.24609375$	$0.12109375=0.24609375-0.125$	« 1011. 111 »
-4	$2^{-4}=0.0625 \leq 0.12109375$	$0.05859375=0.12109375-0.0625$	« 1011. 1111 »
-5	$2^{-5}=0.03125 \leq 0.05859375$	$0.02734375=0.05859375-0.03125$	« 1011. 11111 »
-6	$2^{-6}=0.015625 \leq 0.02734375$	$0.01171875=0.02734375-0.015625$	« 1011. 111111 »
-7	$2^{-7}=0.0078125 \leq 0.01171875$	$0.00390625=0.01171875-0.0078125$	« 1011. 1111111 »
-8	$2^{-8}=0.00390625 \leq 0.00390625$	$0=0.00390625-0.00390625$	« 1011. 11111111 »





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

- $N = 11.99609375_{10}$ est donc égal à 1011.11111111_2
- Nous devons maintenant normaliser ce nombre, nous obtenons donc $N = 1.011111111111_2 \times 2^3_{10}$
- Finalement, le codage de N selon le format IEEE 754 est :
 - Bit de signe à 0 (le nombre est positif)
 - Champ exposant = $3_{10} + 127_{10}$ soit 10000010_2
 - Champ mantisse = $011111111110000000000000_2$
- On obtient donc **$0\ 10000010\ 011111111110000000000000_2$**





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

■ $N = 7.33203125_{10}$ avec $mMax=3_{10}$

M	Test	N	B
départ		7.33203125	« »
3	$2^3=8 > 7.33203125$	7.33203125	« 0 »
2	$2^2=4 \leq 7.33203125$	$3.33203125=7.33203125-4$	« 01 »
1	$2^1=2 \leq 3.33203125$	$1.33203125=3.33203125-2$	« 011 »
0	$2^0=1 \leq 1.33203125$	$0.33203125=1.33203125-1$	« 0111. »
-1	$2^{-1}=0.5 > 0.33203125$	0.33203125	« 0111.0 »
-2	$2^{-2}=0.25 \leq 0.33203125$	$0.08203125=0.33203125-0.25$	« 0111.01 »
-3	$2^{-3}=0.125 > 0.08203125$	0.08203125	« 0111.010 »
-4	$2^{-4}=0.0625 \leq 0.08203125$	$0.01953125=$ $0.08203125-0.0625$	« 0111.0101 »
-5	$2^{-5}=0.03125 > 0.01953125$	0.01953125	« 0111.01010 »
-6	$2^{-6}=0.015625 \leq 0.01953125$	$0,00390625=$ $0.01953125-0.015625$	« 0111.010101 »
-7	$2^{-7}=0.0078125$ et $0.0078125 > 0.00390625$	0.0039062	« 0111.0101010 »
-8	$2^{-8}=0.00390625$ et $0.00390625 \leq 0.00390625$	$0=0.0039062-0.00390625$	« 0111.01010101 »





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

- $N = 7.33203125_{10}$ est donc égal à 0111.01010101_2
- Nous devons maintenant normaliser ce nombre, nous obtenons donc $N = 1.1101010101 \times 2^2$
- Finalement, le codage de N selon le format IEEE 754 est :
 - Bit de signe à 0 (le nombre est positif)
 - Champ exposant = $2_{10} + 127_{10}$ soit 10000001_2
 - Champ mantisse = $110101010100000000000000_2$
- On obtient donc **$0\ 10000001\ 110101010100000000000000_2$**





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754

- Si le nombre réel est déjà écrit en notation scientifique en base (de la forme mantisse $\times 10^{\text{exposant}}$) utiliser une procédure modifiée
 - conversion de la mantisse écrite en base 10, vers la base 2 en utilisant la même version modifiée de **l'algorithme d'écriture**
 - conversion du nombre 10^n en base 2
 - multiplier les deux nombres binaires obtenus
 - **normalisation** du résultat
 - troncature éventuelle de la nouvelle mantisse selon la précision choisie





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754

■ Comment passer de 10^n_{10} à $m_2 2^e_{10}$

■ On a $N = 10^n_{10} = (8_{10} + 2_{10})^n = (2^3_{10} + 2^1_{10})^n$

■ On doit utiliser la formule du **binôme de Newton**, on obtient donc :

$$N = (a + b)^n = \sum_{p=0}^n C_n^p \cdot a^{n-p} \cdot b^p \text{ avec } \begin{cases} C_n^p = \frac{n!}{(n-p)! \times p!} \\ a = 2^3 \text{ et } b = 2^1 \end{cases}$$

$$N = \sum_{p=0}^n C_n^p \cdot 2^{3(n-p)} \cdot 2^p = \sum_{p=0}^n C_n^p \cdot 2^{3(n-p)+p} = \sum_{p=0}^n C_n^p \cdot 2^{3n-2p}$$

$$N = \left(\sum_{p=0}^n C_n^p \cdot 2^{-2p} \right) \cdot 2^{3n}$$





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

- On considère le nombre $N = 7.33203125_{10} \times 10^3_{10}$

- On divise ce nombre en deux parties :
 - $N' = 7.33203125_{10}$
 - $N'' = 10^3_{10}$

- On convertit ces deux nombres en binaires.





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

■ $N' = 7.33203125_{10}$ (même calcul que précédemment)

M	Test	N'	B
départ		7.33203125	« »
3	$2^3=8 > 7.33203125$	7.33203125	« 0 »
2	$2^2=4 \leq 7.33203125$	$3.33203125=7.33203125-4$	« 01 »
1	$2^1=2 \leq 3.33203125$	$1.33203125=3.33203125-2$	« 011 »
0	$2^0=1 \leq 1.33203125$	$0.33203125=1.33203125-1$	« 0111. »
-1	$2^{-1}=0.5 > 0.33203125$	0.33203125	« 0111.0 »
-2	$2^{-2}=0.25 \leq 0.33203125$	$0.08203125=0.33203125-0.25$	« 0111.01 »
-3	$2^{-3}=0.125 > 0.08203125$	0.08203125	« 0111.010 »
-4	$2^{-4}=0.0625 \leq 0.08203125$	$0.01953125=$ $0.08203125-0.0625$	« 0111.0101 »
-5	$2^{-5}=0.03125 > 0.01953125$	0.01953125	« 0111.01010 »
-6	$2^{-6}=0.015625 \leq 0.01953125$	$0.00390625=$ $0.01953125-0.015625$	« 0111.010101 »
-7	$2^{-7}=0.0078125$ et $0.0078125 > 0.00390625$	0.00039062	« 0111.0101010 »
-8	$2^{-8}=0.00390625$ et $0.00390625 \leq 0.00390625$	$0=0.0039062-0.00390625$	« 0111.01010101 »





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

■ $N'' = 10^3_{10} = (8_{10} + 2_{10})^3 = (2^3_{10} + 2^1_{10})^3$

■ En utilisant la formule déterminée précédemment, on a :

■
$$N'' = \left(\sum_{p=0}^n C_n^p \cdot 2^{-2p} \right) \cdot 2^{3n}$$

■
$$N'' = (1 \times 2^{-0} + 3 \times 2^{-2} + 3 \times 2^{-4} + 1 \times 2^{-6}) \times 2^{3 \times 3}$$

■
$$N'' = 2^9 + 3 \times 2^7 + 3 \times 2^5 + 2^3$$

■
$$N'' = 2^9 + 2 \times 2^7 + 2^7 + 2 \times 2^5 + 2^5 + 2^3$$

■
$$N'' = 2^9 + 2^8 + 2^7 + 2^6 + 2^5 + 2^3$$

■
$$N'' = 1111101000_2$$





Les binaires à virgule flottante

Conversion décimal réel vers binaire IEEE 754 – Exemple

- $N' = 7.33203125_{10}$ est donc égal à 0111.01010101_2
- $N'' = 10^3$ est égal à 1111101000_2
- Si nous effectuons la multiplication binaire, nous obtenons $N = 1110010100100.00001000_2 = 1.11001010010000001 \times 2^{12}$
- Finalement, le codage de N' selon le format IEEE 754 est :
 - Bit de signe à 0 (le nombre est positif)
 - Champ exposant = $12 + 127$ soit 10001011
 - Champ mantisse = 11001010010000001000000
- On obtient donc **$0\ 10001011\ 11001010010000001000000_2$**





IEEE 754 – Configurations particulières

Le codage de l'infini

- Pour coder l'infini, il faut remplir le champ exposant avec des 1 et le champ mantisse avec des 0, on a alors :
 - moins l'infini ($-\infty$) si le bit de signe est à 1

Simple précision	1	11111111	000000000000000000000000
Double précision	1	111111111111	000000000000000000000000 0000
Précision étendue	1	1111111111111111	000000000000000000000000 000000

- plus l'infini ($+\infty$) si le bit de signe est à 0

Simple précision	0	11111111	000000000000000000000000
Double précision	0	111111111111	000000000000000000000000 0000
Précision étendue	0	1111111111111111	000000000000000000000000 000000





IEEE 754 – Configurations particulières

Le codage du zéro

- La notation scientifique ne permet pas d'écrire un « vrai » 0.
- Pour coder le zéro, il faut remplir le champ exposant avec des 0 et le champ mantisse avec des 0, on a alors :
 - moins zéro (-0) si le bit de signe est à 1

Simple précision	1	00000000	000000000000000000000000
Double précision	1	000000000000	000000000000000000000000 0000
Précision étendue	1	0000000000000000	000000000000000000000000 000000

- plus l'infini ($+0$) si le bit de signe est à 0

Simple précision	0	00000000	000000000000000000000000
Double précision	0	000000000000	000000000000000000000000 0000
Précision étendue	0	0000000000000000	000000000000000000000000 000000





IEEE 754 – Configurations particulières

Le codage de l'indécision

- Si une opération ne peut pas retourner un nombre dans R, il faut retourner un code particulier
- Ce code est appelé « **NaN** » pour « **Not a Number** »
 - le bit de signe n'est pas significatif (mais souvent à 1)
 - tous les bits du champ exposant sont à 1
 - le champ mantisse est différent de 0 (mais tous les bits sont souvent à 1)

Simple précision	1	11111111	111111111111111111111111
Double précision	1	111111111111	111111111111111111111111 1111
Précision étendue	1	1111111111111111	111111111111111111111111 111111





IEEE 754 – Configurations particulières

Les nombres dénormalisés

- On utilise les nombres dénormalisés pour coder les petites valeurs
- Le champ exposant ne contient que des 0 (l'exposant réel est donc égal à $-\text{biais}$) et le champ mantisse est différent de 0.
- Pour convertir un nombre dénormalisé en base 10, on doit utiliser la formule ci-dessous :

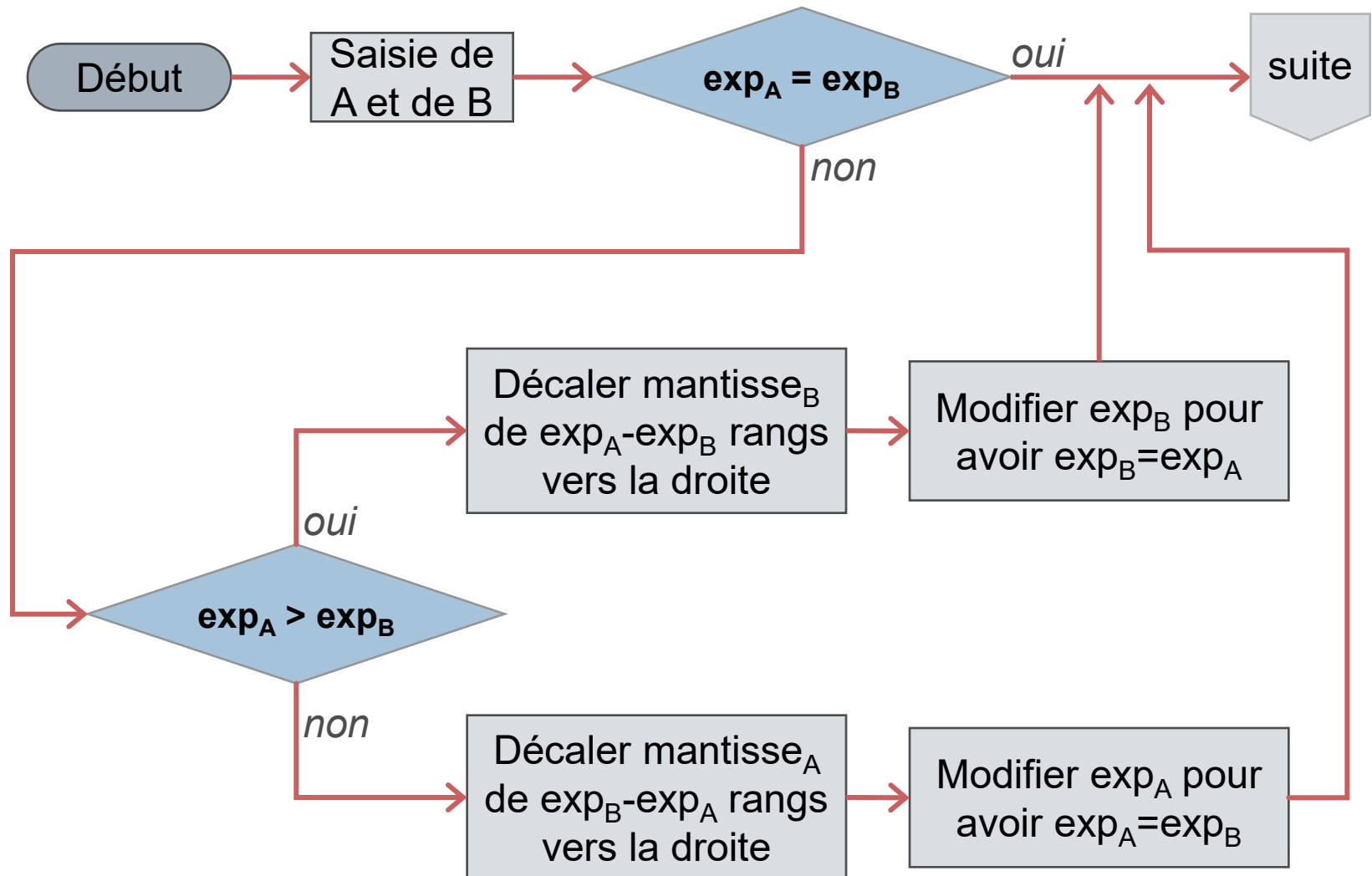
$$N = (-1)^{\text{signe}} \times \left(0 + \sum_{i=1}^{\text{taille}(\text{mantisse})} \text{mantisse}_i \times 2^{-i} \right) \times 2^{-\text{biais}}$$





IEEE 754 – Addition et soustraction

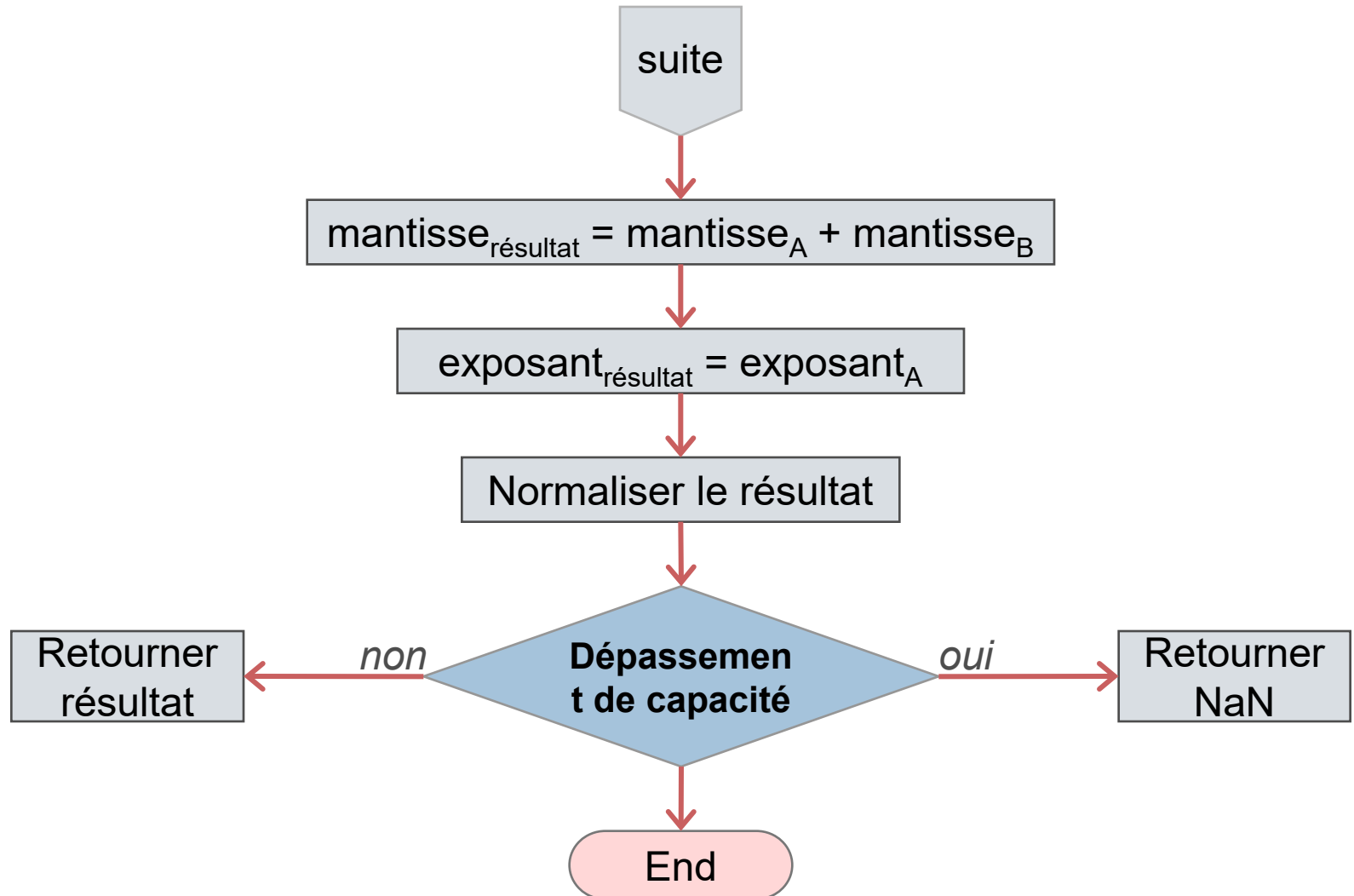
Addition de deux binaires IEEE 754 – Le principe





IEEE 754 – Addition et soustraction

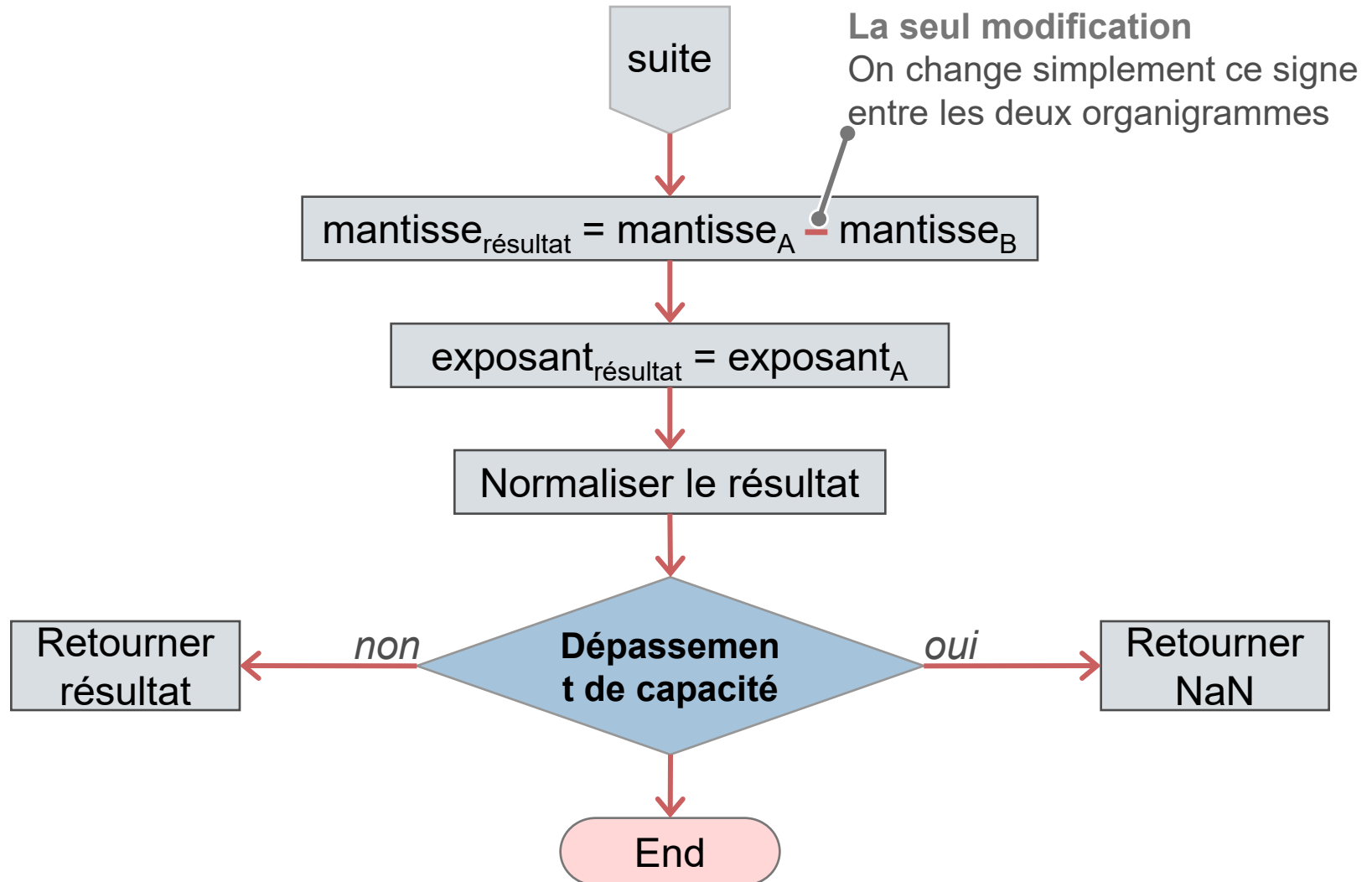
Addition de deux binaires IEEE 754 – Le principe





IEEE 754 – Addition et soustraction

Soustraction de deux binaires IEEE 754 – Même principe





IEEE 754 – Addition et soustraction

Addition de deux binaires IEEE 754 – Exemple

- Additionner $0\ 10000010\ 001000000000000000000000_2$
et $0\ 10000011\ 000100000000000000000000_2$

Calcul en base 2 selon le format IEEE 754

	0	10000011	0.1001000000000000000000000000
+	0	10000011	1.0001000000000000000000000000
	0	10000011	1.1010000000000000000000000000
	↓	↓	↓
	0	10000011	1010000000000000000000000000

Calcul en
base 10

	9
+	17
	26





IEEE 754 – Addition et soustraction

Addition de deux binaires IEEE 754 – Exercice

- Additionner $0\ 01111110\ 100000000000000000000000_2$ et $0\ 10000101\ 100100010000000000000000_2$

Calcul en base 2 selon le format IEEE 754

Calcul en base 10

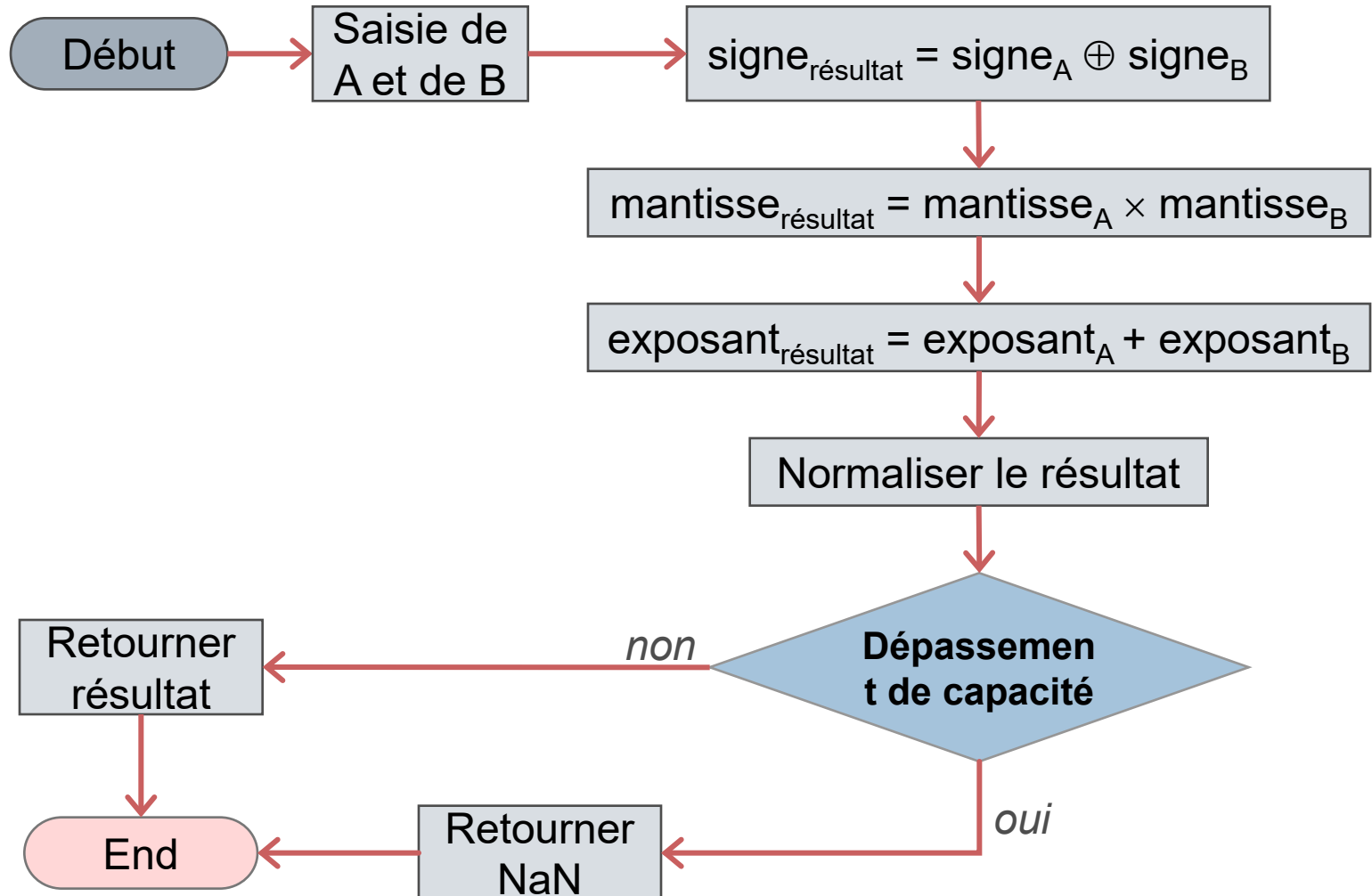
	0	10000101	0.00000011000000000000000000000000	0.75
+	0	10000101	1.10010001000000000000000000000000	+ 100.25
	0	10000101	1.10010100000000000000000000000000	101.00
	0	10000101	10010100000000000000000000000000	





IEEE 754 – Multiplication et division

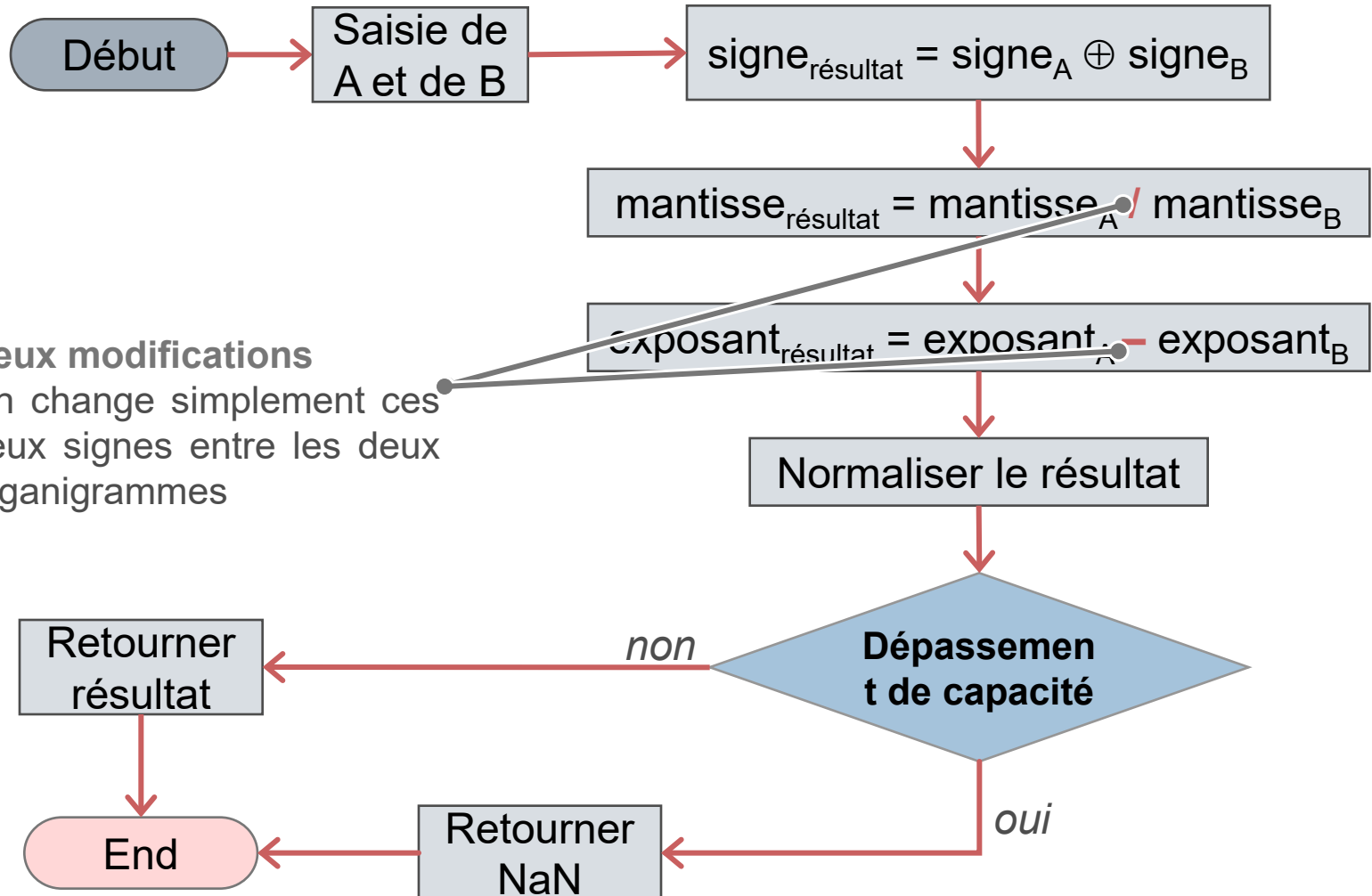
Multiplication de deux binaires IEEE 754 – Le principe





IEEE 754 – Multiplication et division

Multiplication de deux binaires IEEE 754 – Le principe





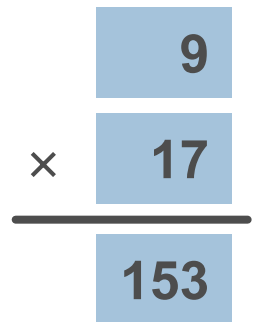
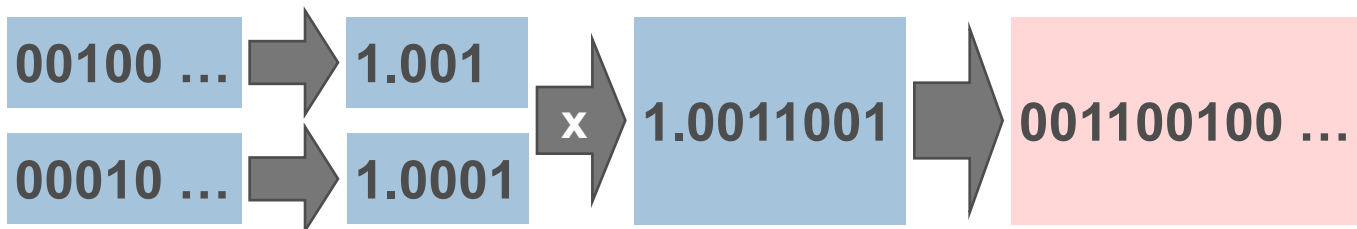
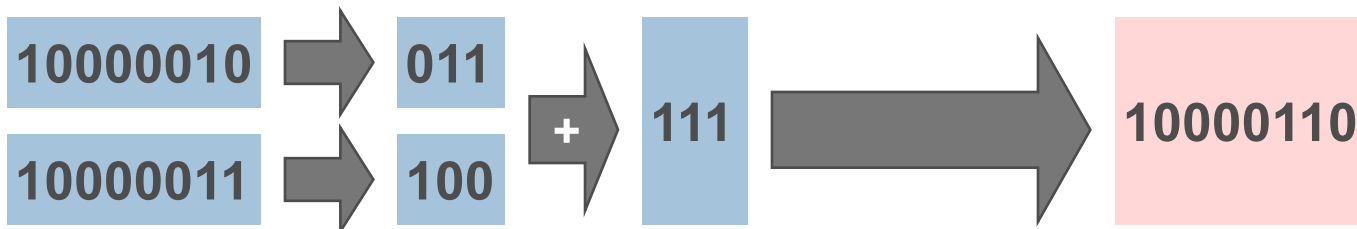
IEEE 754 – Multiplication et division

Multiplication de deux binaires IEEE 754 – Exemple

- Multiplier $0\ 10000010\ 001000000000000000000000_2$ et $0\ 10000011\ 000100000000000000000000_2$

Calcul en base 2 selon le format IEEE 754

Calcul en base 10





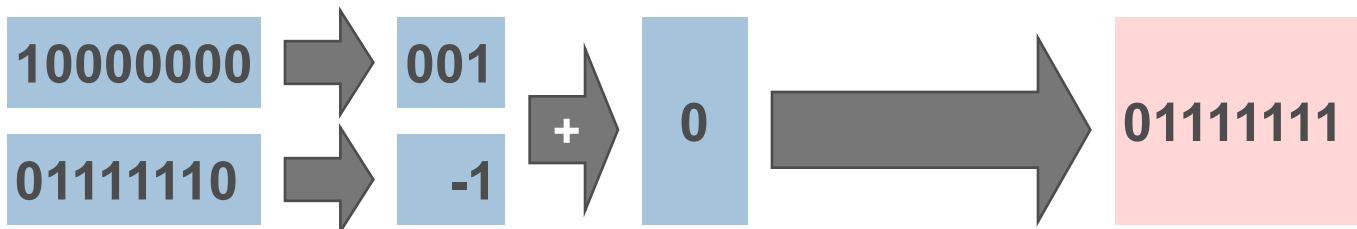
IEEE 754 – Multiplication et division

Multiplication de deux binaires IEEE 754 – Exemple

- Multiplier $0\ 10000000\ 010000000000000000000000_2$
et $0\ 01111110\ 000000000000000000000000_2$

Calcul en base 2 selon le format IEEE 754

Calcul en
base 10





IEEE 754 – Multiplication et division

Multiplication de deux binaires IEEE 754 – Exemple

- Multiplier $1\ 10000010\ 010000000000000000000000_2$
et $0\ 01111110\ 100000000000000000000000_2$

Calcul en base 2 selon le format IEEE 754

Calcul en
base 10

$1 \oplus 0$



1

10000010



011



10



10000001

01111110



-1



10



10000001

01000 ...



1.01



1.111



11100000 ...

10000 ...



1.1



1.111



11100000 ...

×

-10

0.75

-7.5





Le codage hexadécimal

Le principe

- « L'hexa » permet de manipuler des nombres en base 16.
- Nous disposons :
 - des 10 chiffres (0 jusqu'à 9)
 - des 6 premières lettres (A, B, C, D, E et F).
- Ce codage est largement utilisé aujourd'hui car, il permet de manipuler les octets comme des blocs de chiffres hexadécimaux, ce qui évite quelques erreurs de calcul.
- Les opérations sur ce type de base suivent les principes énoncés précédemment pour les binaires.





Le codage hexadécimal

Table de correspondance

Décimal	Binaire	Hexa
0	000000	00
1	000001	01
2	000010	02
3	000011	03
4	000100	04
5	000101	05
6	000110	06
7	000111	07
8	001000	08
9	001001	09
10	001010	0A
11	001011	0B
12	001100	0C
13	001101	0D
14	001110	0E
15	001111	0F
16	010000	10

Décimal	Binaire	Hexa
17	010001	11
18	010010	12
19	010011	13
20	010100	14
21	010101	15
22	010110	16
23	010111	17
24	011000	18
25	011001	19
26	011010	1A
27	011011	1B
28	011100	1C
29	011101	1D
30	011110	1E
31	011111	1F
32	100000	20
33	100001	21





Le codage hexadécimal

Conversion hexadécimal vers décimal – Exemple

- A l'instar du codage binaire, il est possible de déterminer la valeur décimale associée à un nombre hexadécimal en utilisant un tableau de correspondance.

Nombre hexa.	1	A	F	0	C						
	↓	↓	↓	↓	↓						
Valeurs	1	10	15	0	12						
	×	×	×	×	×						
Coefficients	65536	4096	256	16	1						
	↓	↓	↓	↓	↓						
Nombre décimal	65536	+	40960	+	3840	+	0	+	12	=	110348





Le codage hexadécimal

Conversion hexadécimal vers décimal – Exercice

■ Traduire $2B7F4_{16}$ en décimal

Nombre hexa.	2	B	7	F	4						
	↓	↓	↓	↓	↓						
Valeurs	2	11	7	15	4						
	×	×	×	×	×						
Coefficients	65536	4096	256	16	1						
	↓	↓	↓	↓	↓						
Nombre décimal	131072	+	45056	+	1792	+	240	+	4	=	178164





Le codage hexadécimal

Conversion décimal vers hexadécimal – Exemple

- Comme pour le codage binaire, il est également possible de convertir un nombre décimal en hexadécimal grâce à une suite de division par 16.
- Si on considère la conversion du nombre 2765_{10} , on a :

$$\begin{array}{r|l} 2765 & 16 \\ \hline -2752 & 172 \\ \hline 13 & \end{array} \quad \begin{array}{r|l} 172 & 16 \\ \hline -160 & 10 \\ \hline 12 & \end{array} \quad \begin{array}{r|l} 10 & 16 \\ \hline -0 & 0 \\ \hline 10 & \end{array}$$

- Si on lit de droite à gauche, on 10, 12 et 13 soit ACD_{16}





Le codage hexadécimal

Conversion décimal vers hexadécimal – Exercice

- Traduire 3107_{10} en hexadécimal

$$\begin{array}{r|l} 3107 & 16 \\ \hline -3104 & 194 \\ \hline 3 & \end{array} \quad \begin{array}{r|l} 194 & 16 \\ \hline -192 & 12 \\ \hline 2 & \end{array} \quad \begin{array}{r|l} 12 & 16 \\ \hline -0 & 0 \\ \hline 12 & \end{array}$$

- En lisant de droite à gauche, on a 12, 2 et 3 soit $C23_{16}$.

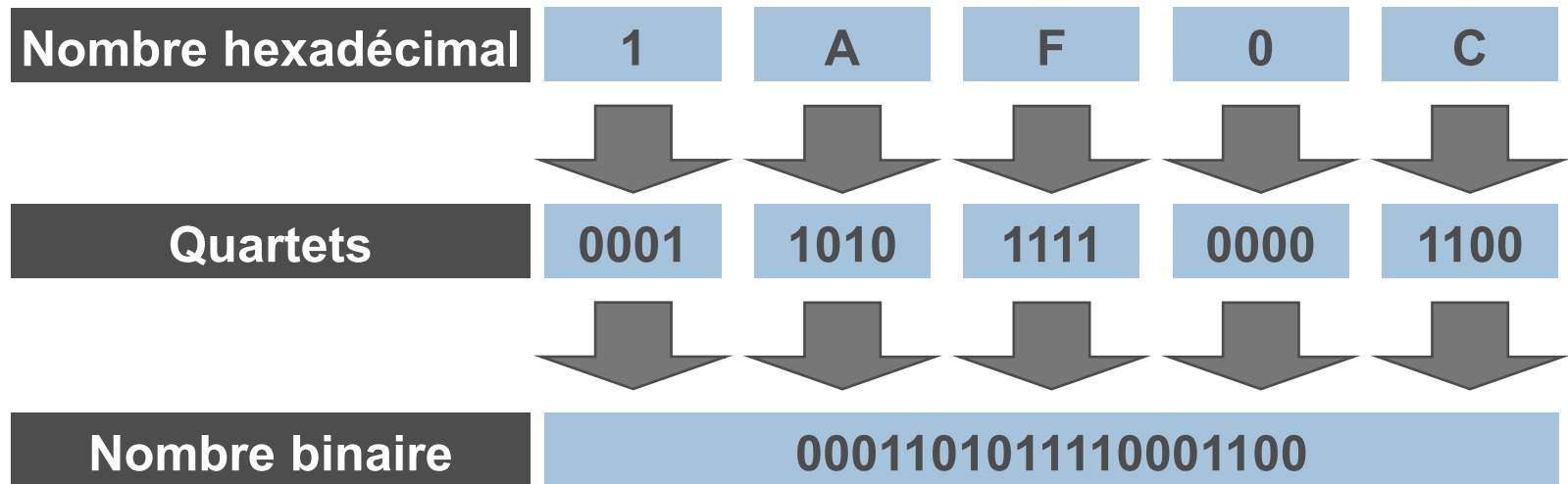




Le codage hexadécimal

Conversion hexadécimal vers binaire – Exemple

- La conversion hexadécimal vers binaire s'effectue très simplement car un nombre hexadécimal correspond à un bloc de quatre bit (**quartet**).





Le codage hexadécimal

Conversion hexadécimal vers binaire – Exercice

- Traduire $B2DE4_{16}$ en binaire

Nombre hexadécimal	B	2	D	E	4
	↓	↓	↓	↓	↓
Quartets	1011	0010	1101	1110	0010
	↓	↓	↓	↓	↓
Nombre binaire	10110010110111100010				





Le codage hexadécimal

Conversion binaire vers hexadécimal – Exemple

- Pour effectuer la conversion inversion, il suffit de scinder le nombre binaire en quartet (en commençant par les bit de poids faible) puis de convertir chaque quartet pour obtenir le nombre hexadécimal correspondant.

Nombre binaire	101101011010011000				
	↓	↓	↓	↓	↓
Quartets	0010	1101	0110	1001	1000
	↓	↓	↓	↓	↓
Nombre hexadécimal	2	D	6	9	8





Le codage hexadécimal

Conversion binaire vers hexadécimal – Exercice

- Convertir 101001011001011010_2

Nombre binaire	101001011001011010				
Quartets	0010	1001	0110	0101	1010
Nombre hexadécimal	2	9	6	5	A





Le codage octal et le DCB

Le principe du codage octal

- Utilisée pour manipuler des nombres en base 8.
- Nous disposons des chiffres 0, 1, 2, 3, 4, 5, 6 et 7.
- Employé au début de l'informatique car les premières machines travaillaient sur des « mots » qui avaient une largeur de 12 bits.
- Les opérations sur ce type de base suivent les principes énoncés précédemment pour les binaires.





Le codage octal et le DCB

Table de correspondance du codage octal

Décimal	Binaire	Octal
0	00000	00
1	00001	01
2	00010	02
3	00011	03
4	00100	04
5	00101	05
6	00110	06
7	00111	07
8	01000	10
9	01001	11
10	01010	12
11	01011	13
12	01100	14
13	01101	15
14	01110	16
15	01111	17
16	10000	20





Le codage octal et le DCB

Le principe du DCB

- Le « Décimal Codé Binaire » (BCD ou Binary Coded Decimal) est un code de représentation des nombres dans les systèmes numériques.
- Dans ce code, chaque chiffre de la représentation décimale est codée sur un groupe de 4 bits :
 - avantage : affichage décimal grandement facilité
 - inconvénient : des combinaisons de bits sont inutilisées





Le codage octal et le DCB

Table de correspondance du DCB

Décimal	DCB
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Combinaisons de bits
non-utilisées dans le DCB





Le codage octal et le DCB

Exemple

- Codage du nombre 1664_{10}

	Valeur codée	Taille
En binaire	110 1000 0000	11
En DCB	0001 0110 0110 0100	16





Le codage des caractères

Le principe de la table ASCII 7 bits

- En 1963, le standard ASCII (American Standard Code for Information Interchange) est créé.
- Il finit par s'imposer et devient :
 - un standard ANSI (le X 3.4) en 1968
 - un standard ISO (International Standard Organisation) en 1973 (ISO 636)
 - un standard V.2 au CCITT (Comité Consultatif International Télégraphique et Téléphonique).
- Le standard ASCII « de base » est codé sur 7 bits donc il ne permet que l'utilisation de 128 caractères différents.





Le codage des caractères

Table de correspondance de la table ASCII 7 bits

0 NUL (null) ou ESP	32 ESP	64 @	96 `
1 SOH (start of heading) ou ☺	33 !	65 A	97 a
2 STX (start of text) ou ☻	34 "	66 B	98 b
3 ETX (end of text) ou ♥	35 #	67 C	99 c
4 EOT (end of transmission) ou ♦	36 \$	68 D	100 d
5 ENQ (enquiry) ou ♣	37 %	69 E	101 e
6 ACK (acknowledge) ou ♠	38 &	70 F	102 f
7 BEL (bell)	39 '	71 G	103 g
8 BS (backspace)	40 <	72 H	104 h
9 TAB (horizontal tab)	41 >	73 I	105 i
10 LF (NL line feed, new line)	42 *	74 J	106 j
11 VT (vertical tab) ou ♂	43 +	75 K	107 k
12 FF (NP form feed, new page) ou ♀	44 ,	76 L	108 l
13 CR (carriage return)	45 -	77 M	109 m
14 SO (shift out) ou ♀	46 .	78 N	110 n
15 SI (shift int) ou *	47 /	79 O	111 o
16 DLE (data link escape) ou ►	48 0	80 P	112 p
17 DC1 (device controle 1) ou ◄	49 1	81 Q	113 q
18 DC2 (device controle 2) ou ‡	50 2	82 R	114 r
19 DC3 (device controle 3) ou !!	51 3	83 S	115 s
20 DC4 (device controle 4) ou ¶	52 4	84 T	116 t
21 NAK (negative acknowledge) ou §	53 5	85 U	117 u
22 SYN (synchronous idle) ou =	54 6	86 V	118 v
23 ETB (end of transmission block) ou ‡	55 7	87 W	119 w
24 CAN (cancel) ou ↑	56 8	88 X	120 x
25 EM (end of medium) ou ↓	57 9	89 Y	121 y
26 SUB (substitute) ou →	58 :	90 Z	122 z
27 ESC (escape) ou ←	59 ;	91 [	123 <
28 FS (file separator) ou ⊥	60 <	92 \	124
29 GS (group separator) ou ⇨	61 =	93]	125 >
30 RS (record separator) ou ▲	62 >	94 ^	126 ~
31 US (unit separator) ou ▼	63 ?	95 _	127 Δ





Le codage des caractères

Le 8^{ème} bit de la table ASCII 7 bits

- Le bit non utilisé est utilisé comme un bit de parité (*parity check*) de la manière suivante :
 - nous effectuons l'addition des 7 bits d'informations
 - nous regardons si le bit de poids faible du résultat est à 0 (pair ou *even*) ou à 1 (impair ou *odd*)
 - nous comparons ce bit avec le bit de parité :
 - s'il est indentique, il y a de **bonnes chances** que la transmission soit correcte
 - sinon il existe **au moins** une erreur de parité (un bit a changé de valeur à cause d'un problème électronique)





Le codage des caractères

Le 8^{ème} bit de la table ASCII 7 bits

- Ce mécanisme a été standardisé :
 - par le comité ANSI (USA) en 1976 : ANSI X 3.16
 - au niveau mondial, par le comité CCITT : V.4

- Le bit de parité n'est pas suffisant pour garantir l'intégrité de la transmission et du stockage car **deux changements de parité peuvent leurrer ce mécanisme de détection** mais il a été (est) utilisé pour fiabiliser les communication et le stockage des données (par exemple, barrettes mémoire dotées de bit de parité).





Le codage des caractères

Le principe de la table ASCII 8 bits

- Plus tard, lorsque d'autres méthodes de détection et de correction des erreurs ont été mises en place, le bit de parité de l'ASCII a été utilisé pour stocker de nouveaux symboles.
- Le DOS proposait alors de charger des pages de code supplémentaires adaptés au pays (par exemple, il existe une page de code « latin » qui propose les caractères accentués pour les français).





Le codage des caractères

Une table de caractères supplémentaires en ASCII 8 bits

128	Ç	160	á	192	Ł	224	ó
129	ü	161	í	193	ł	225	ø
130	é	162	ó	194	┐	226	õ
131	â	163	ú	195	└	227	ò
132	ä	164	ñ	196	—	228	õ
133	à	165	Ñ	197	+	229	õ
134	å	166	☉	198	+ ã	230	μ
135	ç	167	☼	199	ã	231	þ
136	œ	168	¿	200	Ã	224	ó
137	è	169	©	201	┌	233	ó
138	ê	170	¬	202	▬	234	ü
139	ï	171	½	203	▬	235	ù
140	î	172	¾	204	▬	236	ý
141	ì	173	↓	205	=	237	ÿ
142	ã	174	«	206	≡	238	˘
143	ä	175	»	207	×	239	˙
144	É	160	▨	208	÷	240	—
145	æ	177	▩	209	ø	241	±
146	œ	178	▪	210	Ð	242	=
147	ô	179	▫	211	Ê	243	¾
148	ö	180	▬	212	È	244	¶
149	ò	181	└	213	È	245	§
150	û	182	└	214	í	246	÷
151	ù	183	└	215	î	247	˘
152	ÿ	184	└	216	ï	248	˙
153	õ	185	└	217	└	249	˙
154	Ü	186	└	218	└	250	˙
155	ø	187	└	219	■	251	1
156	£	188	└	220	■	252	3
157	Ø	189	└	221	■	253	2
158	×	190	└	222	■	254	■
159	ƒ	191	└	223	■	255	ESP

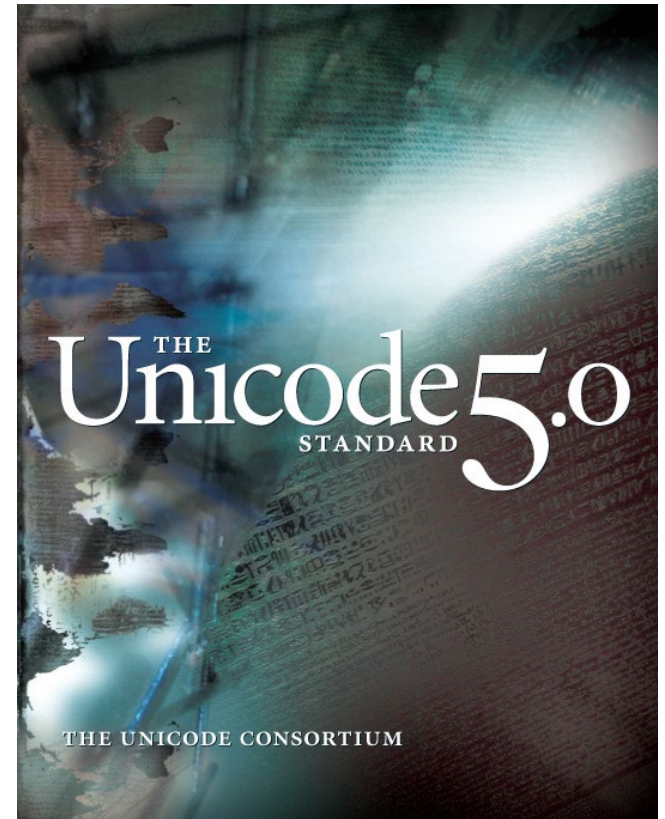




Le codage des caractères

Le principe de l'UNICODE

- Les moyens de stockage se sont nettement améliorés et les ordinateurs sont de plus en plus interconnectés.
- L'**UNICODE**, une nouvelle table de codage sur **16 bits**, a été proposée en 1991.
- Elle contient **65536** caractères permet couvrir les besoins informatiques internationaux.
- Les 128 premiers caractères de cette table correspondent au 128 caractères la table ASCII 7 bits.





Pause-réflexion sur cette 3^{ème} partie

Avez-vous des questions ?





L'algèbre de Boole



Plan de la partie

Voici les chapitres que nous allons aborder:

- Rappels sur la théorie des ensembles.
- Rappels sur le prédicats.
- Synthèse des 4 opérateurs.
- George Boole.
- Caractéristiques de $+$, x , $\bar{}$.
- Principe de dualité et lois de (De) Morgan.
- Analogie entre $\neg$, $\wedge$ et $\vee$, et $+$, x , $\bar{}$.
- L'opérateur $\oplus$.
- Les tables de Karnaugh.





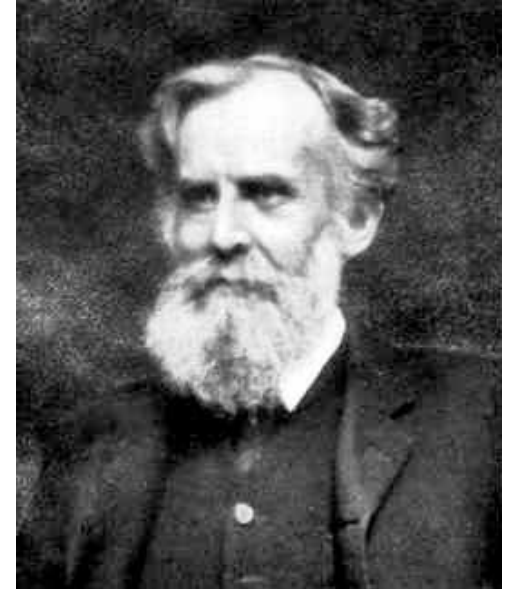
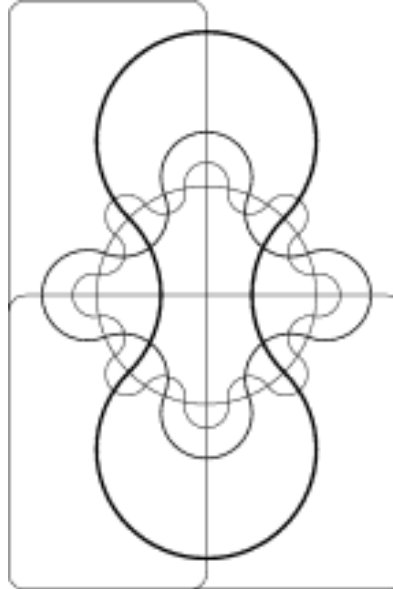
Rappels sur la théorie des ensembles

Théorie des ensembles et diagrammes de Venn

- Théorie des **ensembles** proposé par **Georg Cantor** (1845-1918).
- Représentation des ensembles par des surfaces fermées (et étiquetées) proposée par **John Venn** en 1881.



Georg Cantor



John Venn

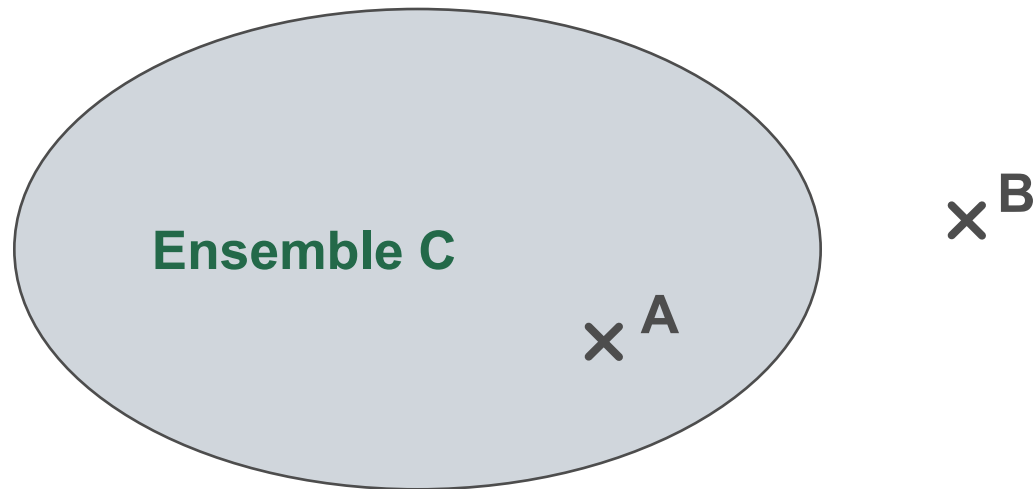




Rappels sur la théorie des ensembles

L'appartenance et la non-appartenance

- les opérateurs d'**appartenance** ($\in$) et de **non-appartenance** ($\notin$) permettent de spécifier qu'un élément fait partie ou ne fait pas partie d'un ensemble.
- Ici, on a $A \in C$ et $B \notin C$.

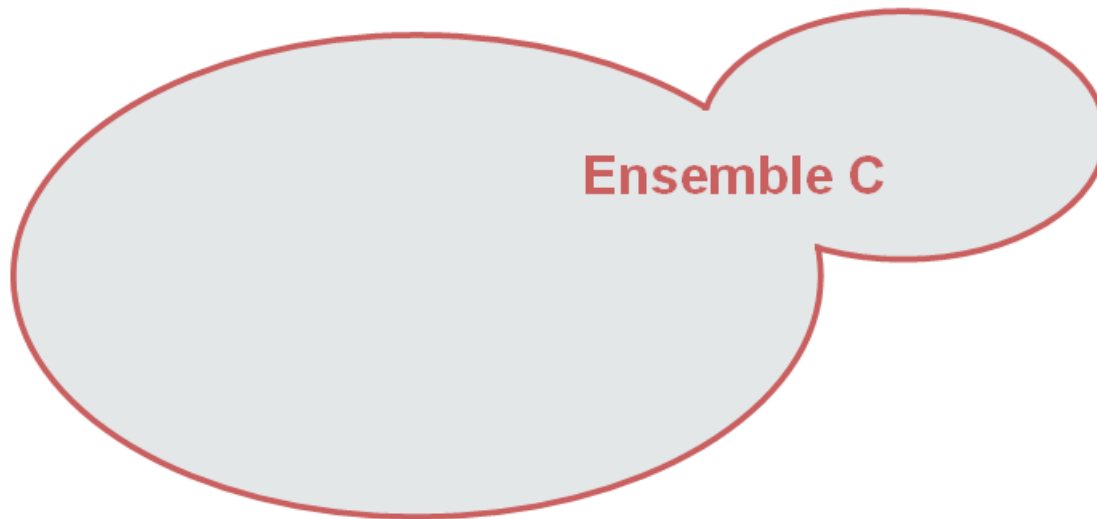




Rappels sur la théorie des ensembles

L'union

- L'opérateur **union** ($\cup$) permet de former un ensemble C en regroupant les éléments des deux ensembles A et B de départ.

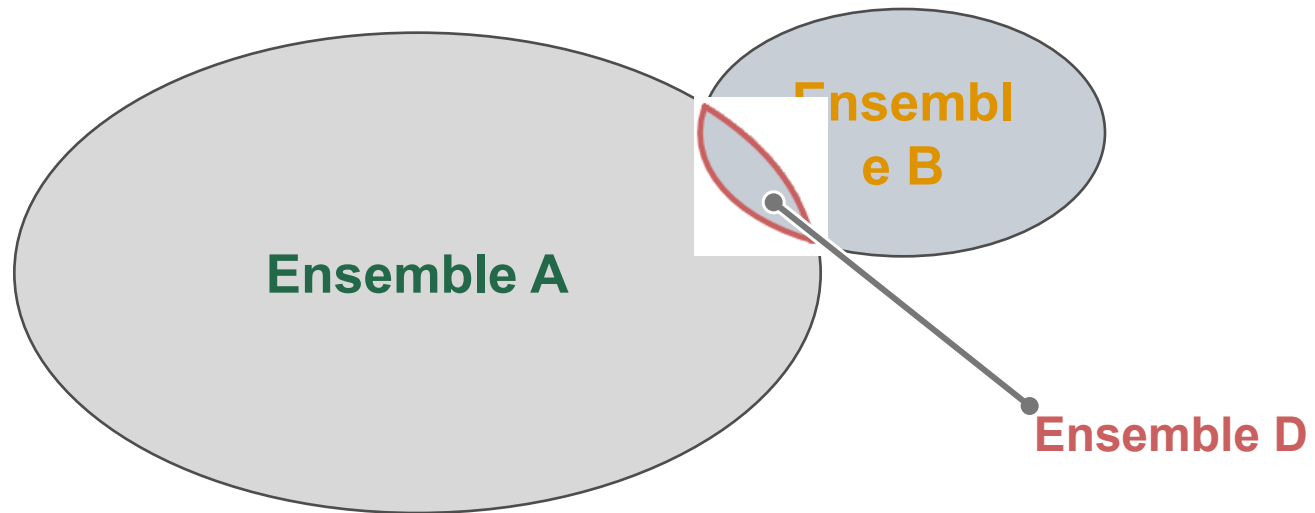




Rappels sur la théorie des ensembles

L'intersection

- L'opérateur **intersection** ($\cap$) permet de former un ensemble D ne contenant que des éléments communs aux deux ensembles A et B de départ.

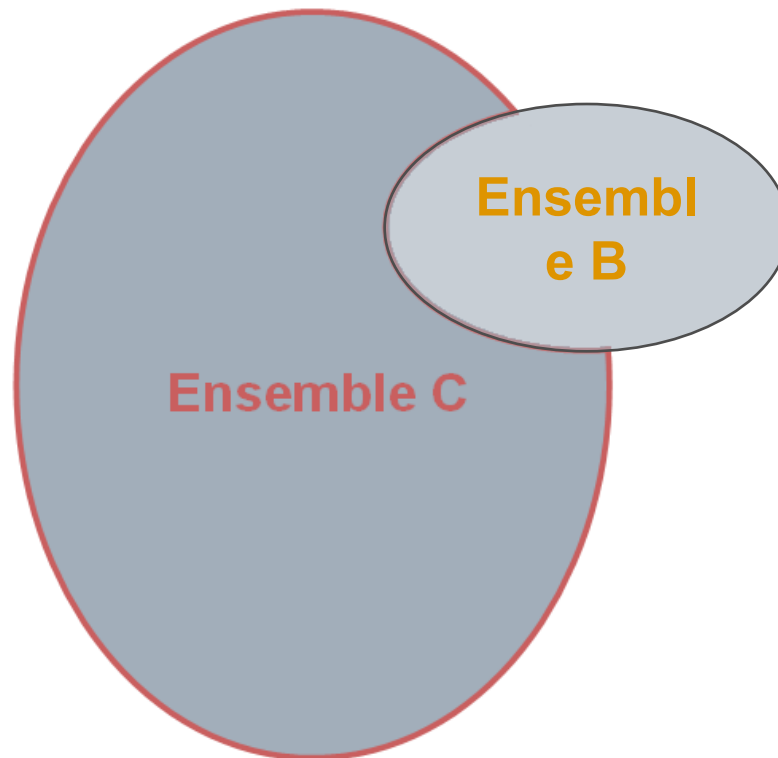




Rappels sur la théorie des ensembles

La soustraction

- L'opérateur **soustraction** ($\setminus$) permet de former un ensemble C en retirant les éléments d'un ensemble B inclus dans un ensemble A.

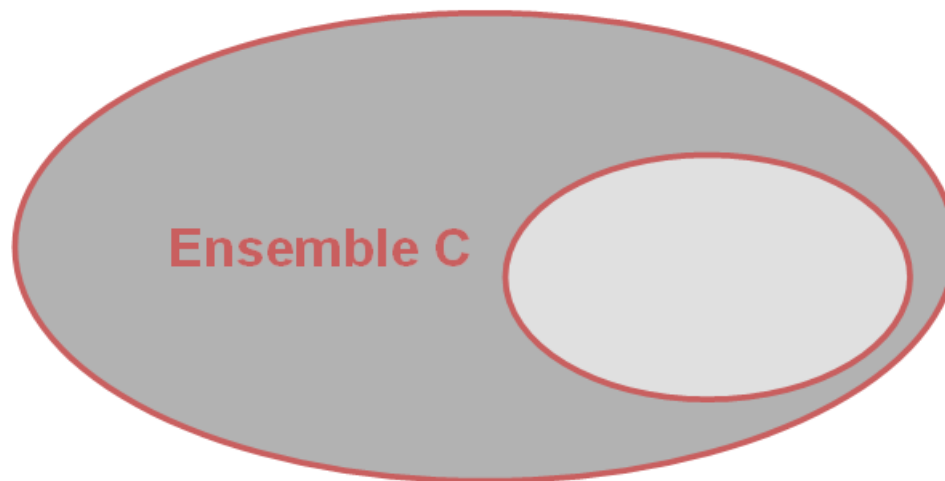




Rappels sur la théorie des ensembles

Le complément

- L'opérateur **complément** (C_A) permet de former un ensemble C contenant les éléments d'un « sur-ensemble » A qui ne sont pas inclus dans B .
- Si nous utilisons le symbole $\bar{}$, nous supposons que l'ensemble A est l'**univers**, c'est-à-dire l'ensemble de tous les éléments que nous manipulons.

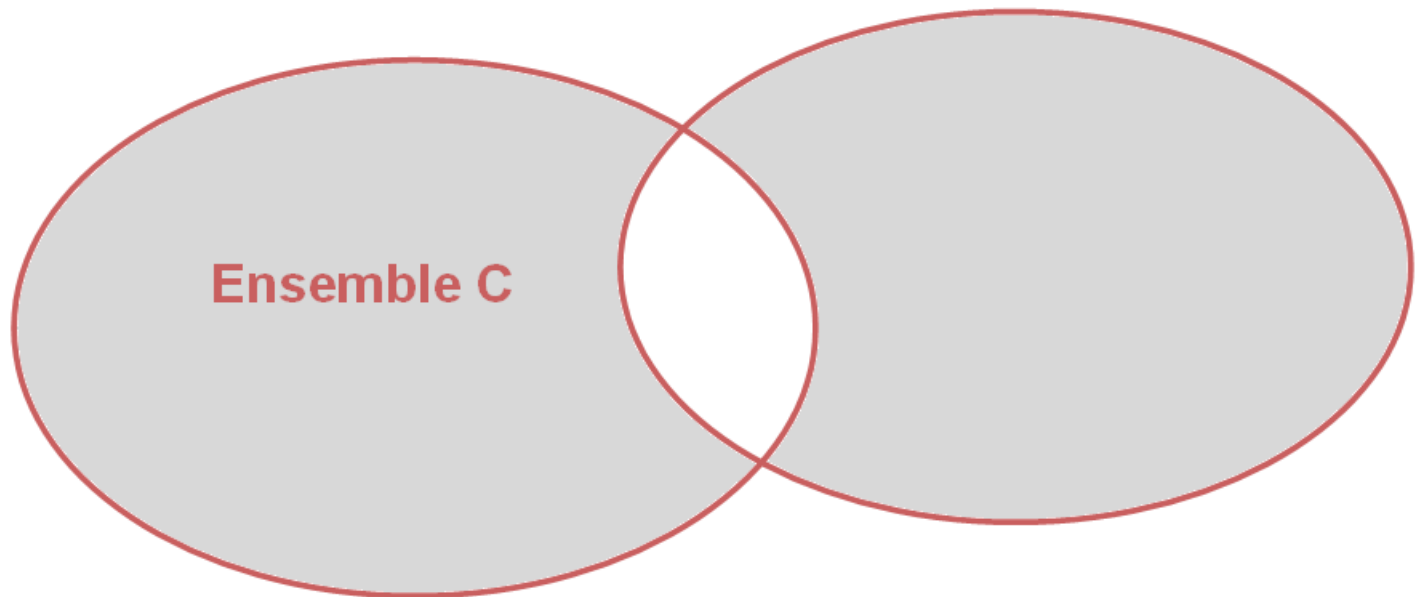




Rappels sur la théorie des ensembles

La différence symétrique

- L'opérateur **différence symétrique** (Δ) permet de former un ensemble C à partir de deux ensembles A et B selon la formule $C = A \Delta B = (A \cup B) \setminus (A \cap B)$.





Rappels sur les prédicats

Introduction

- Un **prédicat** est un test dont le résultat peut prendre deux valeurs : **vrai ou faux**.
- Ces deux valeurs peuvent être codées (en informatique) par **1 et 0**.
- En définitive, l'ensemble $\{0,1\}$ peut être manipulé selon les règles de l'**algèbre binaire** et celles de l'**algèbre de Boole**.

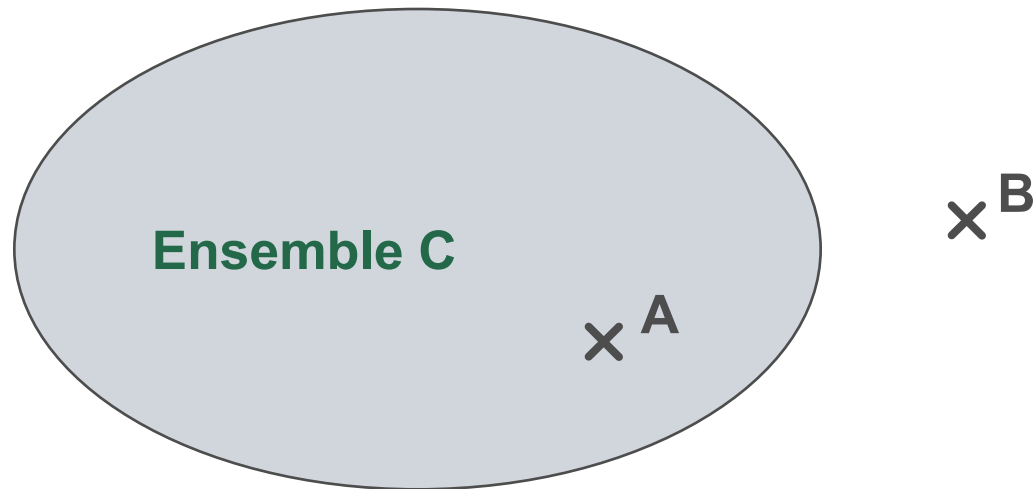




Rappels sur les prédicats

Exemple

- Dans l'exemple ci-dessous, le prédicat $(B \in C)$ est faux alors que $(A \in C)$ est vrai.





Rappels sur les prédicats

L'union et le « ou logique »

- Etant donnée cette nouvelle notion de prédicat, nous pouvons construire un nouvel opérateur appelé « **ou logique** » et noté $\vee$, qui permet de lier deux prédicats de la manière suivante :
 - si au moins l'un des deux prédicats est vrai alors le « ou logique » de ces deux prédicats est vrai
 - si les deux prédicats sont simultanément faux alors le « ou logique » de ces prédicats est également faux

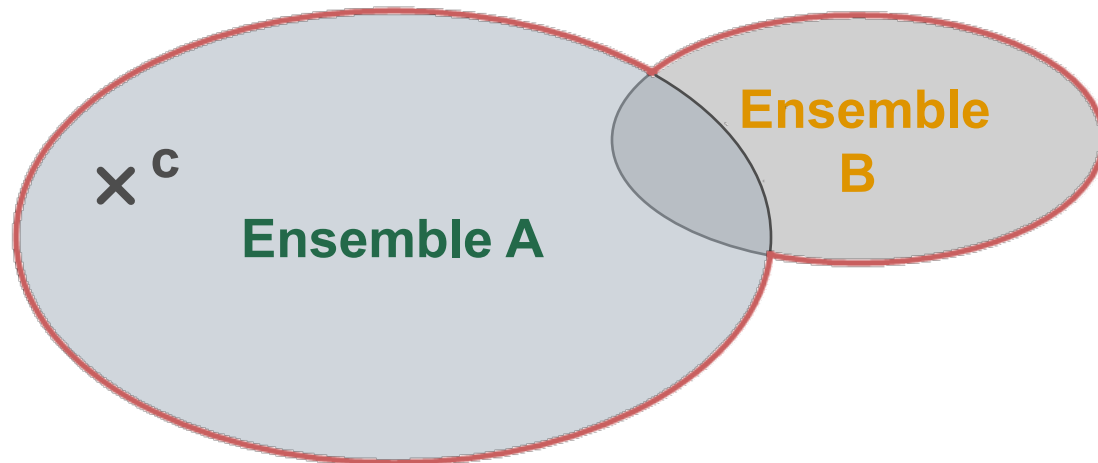




Rappels sur les prédicats

L'union et le « ou logique »

- Comme nous pouvons le constater sur le schéma, cet opérateur appliqué aux prédicats est « équivalent » à l'opérateur d'union appliqué aux ensembles.
- $(c \in A)$ est vrai donc :
 - $((c \in A) \vee (c \in B))$ est vrai
 - $(c \in (A \cup B))$ est vrai

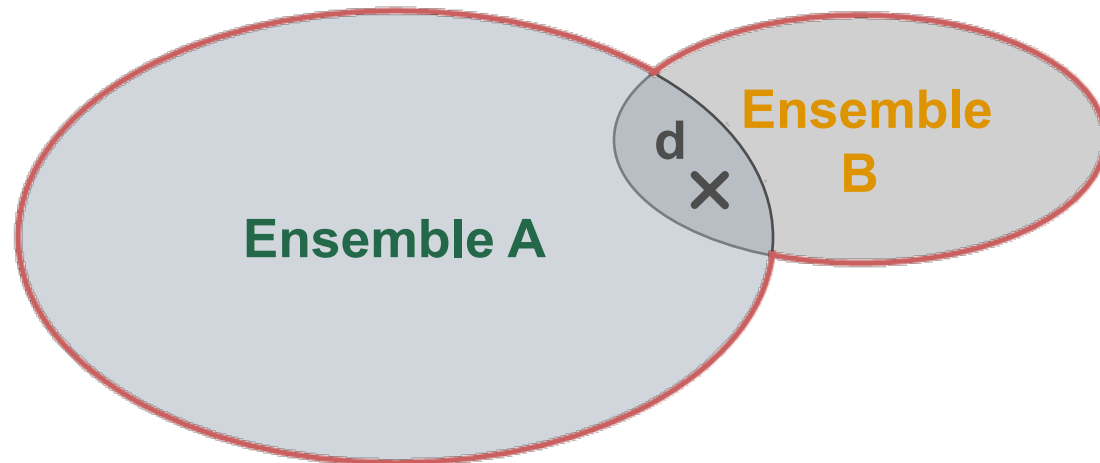




Rappels sur les prédicats

L'union et le « ou logique »

- Comme nous pouvons le constater sur le schéma, cet opérateur appliqué aux prédicats est « équivalent » à l'opérateur d'union appliqué aux ensembles.
- $(d \in A)$ et $(d \in B)$ sont simultanément vrais donc :
 - $((d \in A) \vee (d \in B))$ est vrai
 - $(d \in (A \cup B))$ est vrai

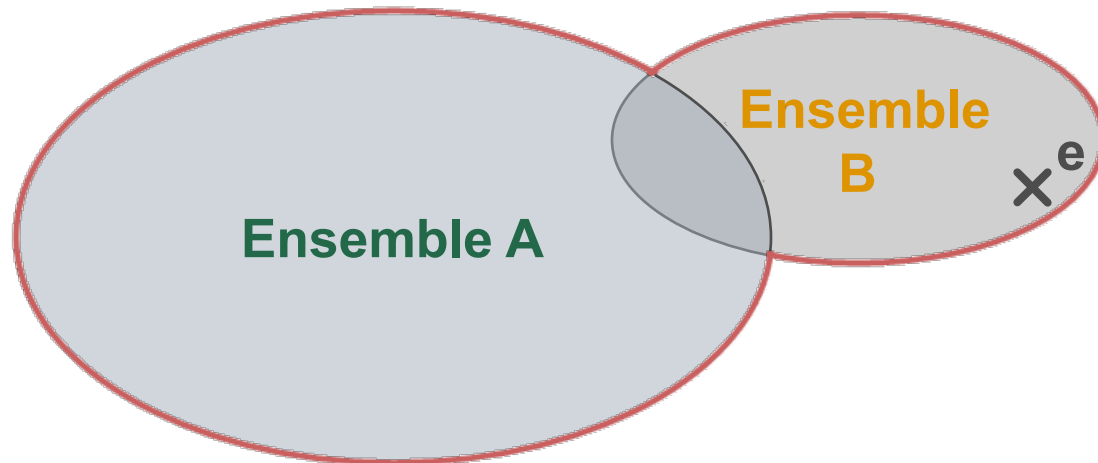




Rappels sur les prédicats

L'union et le « ou logique »

- Comme nous pouvons le constater sur le schéma, cet opérateur appliqué aux prédicats est « équivalent » à l'opérateur d'union appliqué aux ensembles.
- $(e \in B)$ est vrai donc :
 - $((e \in A) \vee (e \in B))$ est vrai
 - $(e \in (A \cup B))$ est vrai

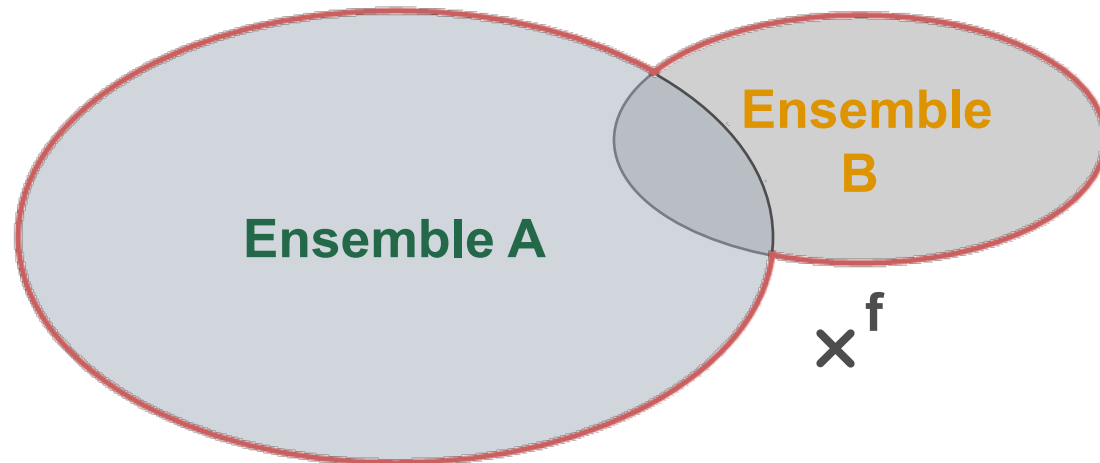




Rappels sur les prédicats

L'union et le « ou logique »

- Comme nous pouvons le constater sur le schéma, cet opérateur appliqué aux prédicats est « équivalent » à l'opérateur d'union appliqué aux ensembles.
- $(f \in A)$ et $(f \in B)$ sont simultanément faux donc :
 - $((f \in A) \vee (f \in B))$ est faux
 - $(f \in (A \cup B))$ est faux





Rappels sur les prédicats

L'intersection et le « et logique »

- A l'instar du « ou logique », nous pouvons créer un opérateur appelé « **et logique** » et noté $\wedge$, qui permet de lier deux prédicats de la manière suivante :
 - si au moins l'un des deux prédicats est faux alors le « et logique » de ces deux prédicats est faux
 - si les deux prédicats sont simultanément vrais alors le « et logique » de ces prédicats est vrai

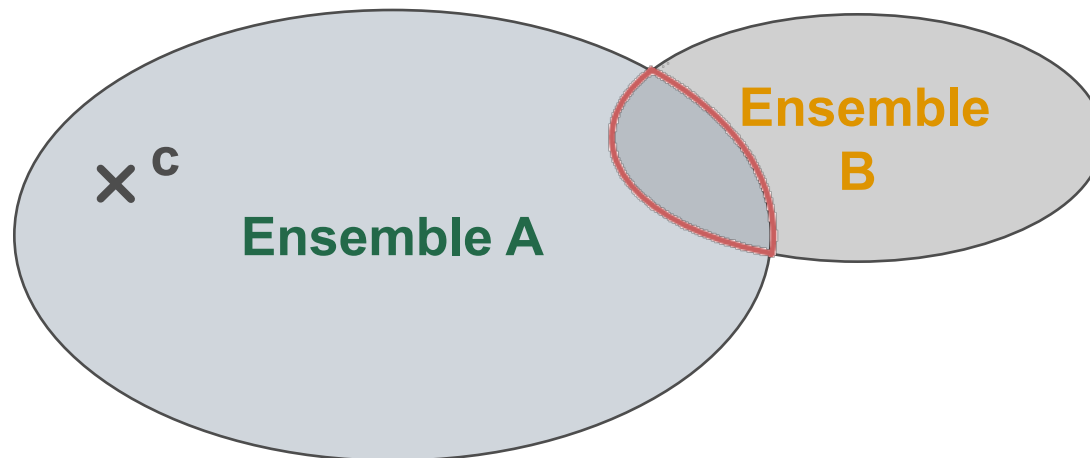




Rappels sur les prédicats

L'intersection et le « et logique »

- Comme précédemment, nous pouvons constater une « équivalence » entre l'opérateur appliqué aux prédicats et l'opérateur d'intersection appliqué aux ensembles
- $(c \in A)$ est vrai mais $(c \in B)$ est faux donc :
 - $((c \in A) \wedge (c \in B))$ est faux
 - $(c \in (A \cap B))$ est faux

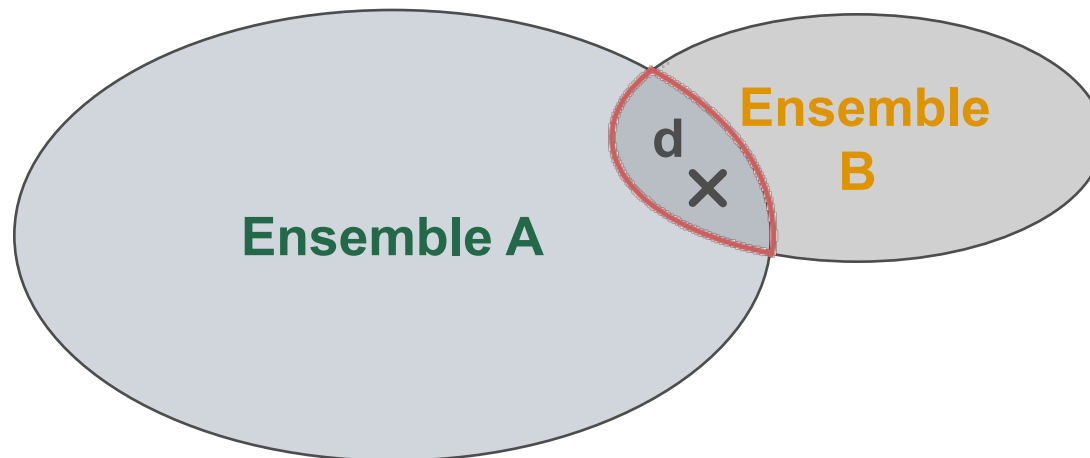




Rappels sur les prédicats

L'intersection et le « et logique »

- Comme précédemment, nous pouvons constater une « équivalence » entre l'opérateur appliqué aux prédicats et l'opérateur d'intersection appliqué aux ensembles
- $(d \in A)$ et $(d \in B)$ sont simultanément vrais donc :
 - $((d \in A) \wedge (d \in B))$ est vrai
 - $(d \in (A \cap B))$ est vrai

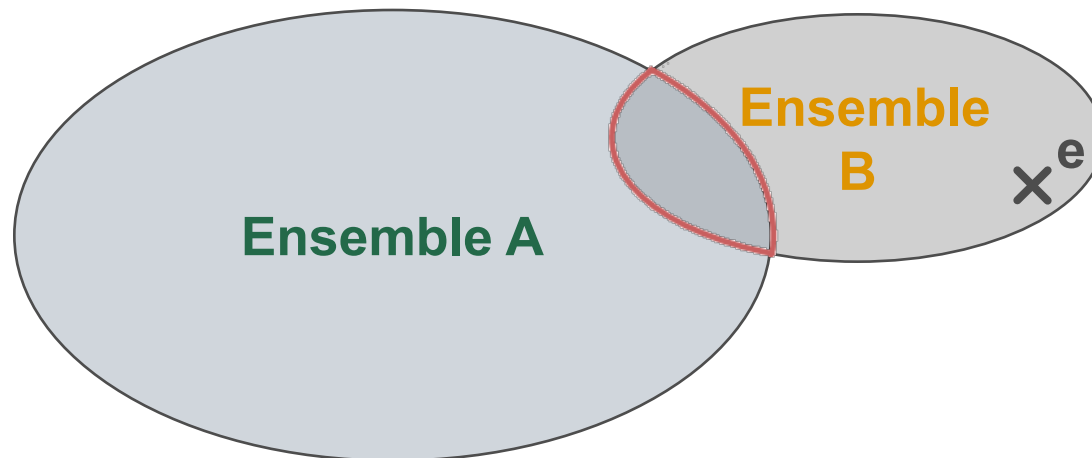




Rappels sur les prédicats

L'intersection et le « et logique »

- Comme précédemment, nous pouvons constater une « équivalence » entre l'opérateur appliqué aux prédicats et l'opérateur d'intersection appliqué aux ensembles
- $(e \in B)$ est vrai mais $(e \in A)$ est faux donc :
 - $((e \in A) \wedge (e \in B))$ est faux
 - $(e \in (A \cap B))$ est faux

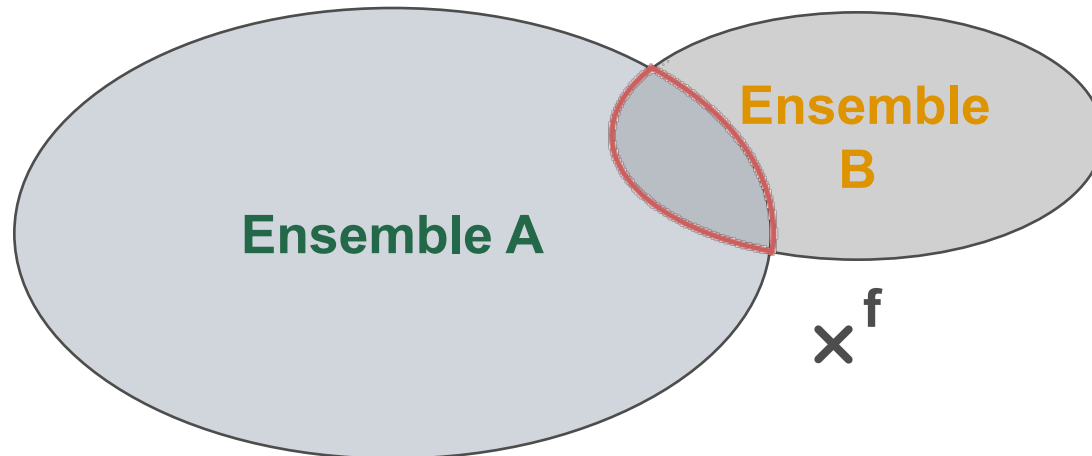




Rappels sur les prédicats

L'intersection et le « et logique »

- Comme précédemment, nous pouvons constater une « équivalence » entre l'opérateur appliqué aux prédicats et l'opérateur d'intersection appliqué aux ensembles
- $(f \in A)$ et $(f \in B)$ sont simultanément faux donc :
 - $((f \in A) \wedge (f \in B))$ est faux
 - $(f \in (A \cap B))$ est faux





Rappels sur les prédicats

La différence symétrique et le « ou exclusif »

- Nous créons un autre opérateur appelé « **ou exclusif** » et noté **ou_e**, qui fonctionne de la manière suivante :
 - si un prédicat est faux et l'autre est vrai alors le « ou exclusif » est vrai
 - si les deux prédicats sont simultanément vrais ou faux alors le « ou exclusif » de ces prédicats est faux

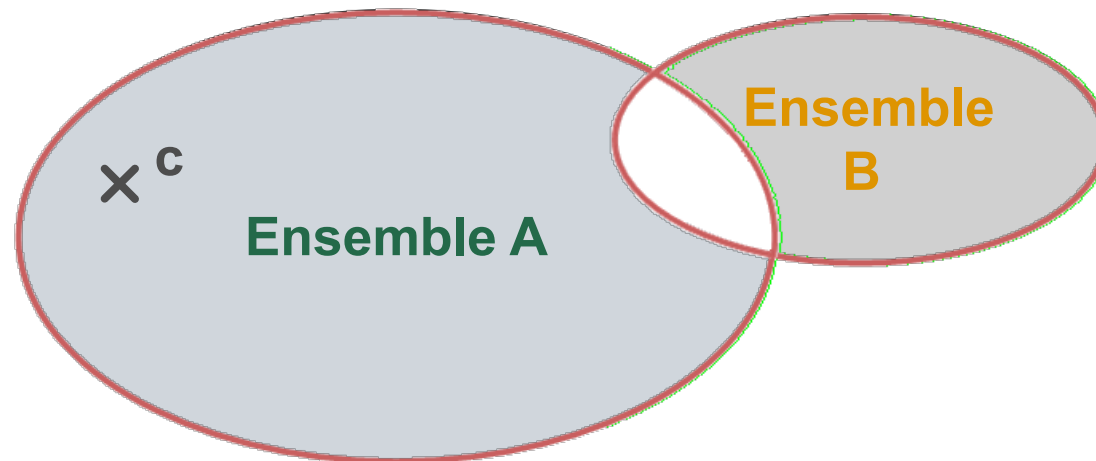




Rappels sur les prédicats

La différence symétrique et le « ou exclusif »

- Cette fois encore, nous constatons une « équivalence » entre le « ou exclusif » et la différence symétrique.
- $(c \in A)$ est vrai et $(c \in B)$ est faux donc :
 - $((c \in A) \text{ ou}_e (c \in B))$ est vrai
 - $(c \in (A \Delta B))$ est vrai

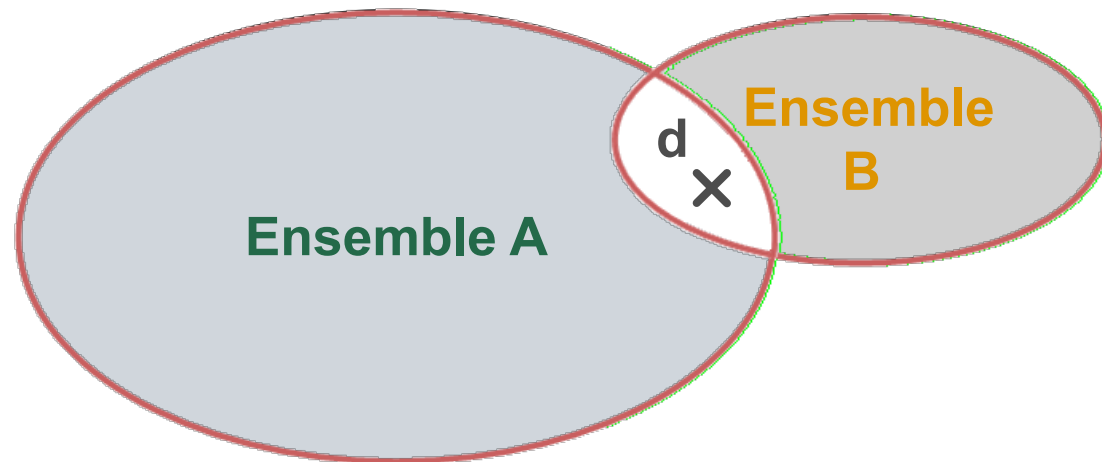




Rappels sur les prédicats

La différence symétrique et le « ou exclusif »

- Cette fois encore, nous constatons une « équivalence » entre le « ou exclusif » et la différence symétrique.
- $(d \in A)$ et $(d \in B)$ sont simultanément vrais donc :
 - $((d \in A) \text{ ou}_e (d \in B))$ est faux
 - $(d \in (A \Delta B))$ est faux

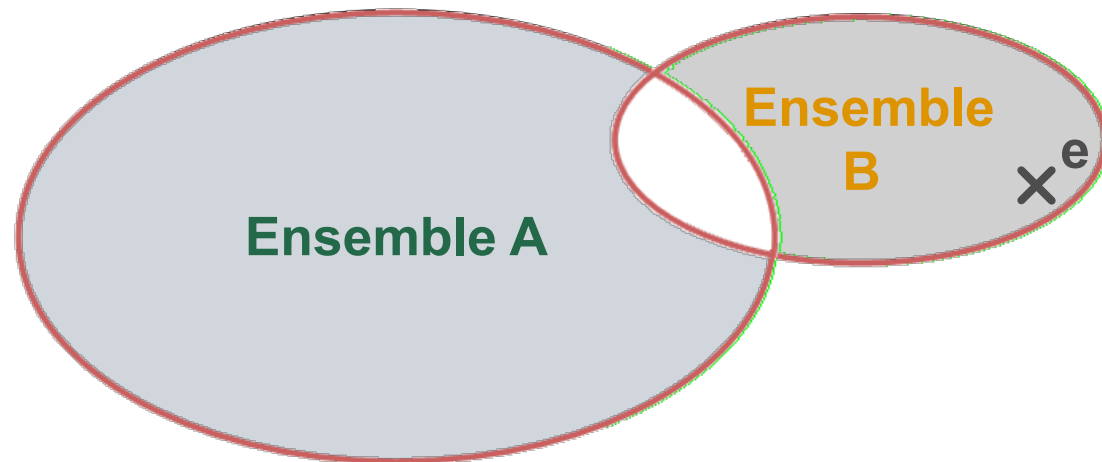




Rappels sur les prédicats

La différence symétrique et le « ou exclusif »

- Cette fois encore, nous constatons une « équivalence » entre le « ou exclusif » et la différence symétrique.
- $(e \in A)$ est faux et $(e \in B)$ est vrai donc :
 - $((e \in A) \text{ ou}_e (e \in B))$ est vrai
 - $(e \in (A \Delta B))$ est vrai

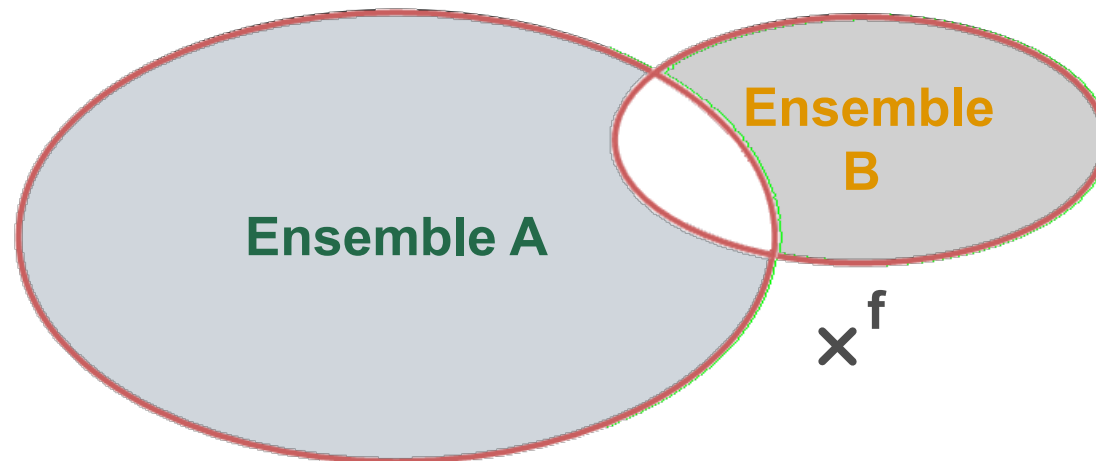




Rappels sur les prédicats

La différence symétrique et le « ou exclusif »

- Cette fois encore, nous constatons une « équivalence » entre le « ou exclusif » et la différence symétrique.
- $(f \in A)$ et $(f \in B)$ sont simultanément faux donc :
 - $((f \in A) \text{ ou}_e (f \in B))$ est faux
 - $(f \in (A \Delta B))$ est faux





Rappels sur les prédicats

Le complément et la négation

- Nous créons un dernier opérateur appelé « **négation** » et noté $\neg$, qui fonctionne de la manière suivante :
 - si un prédicat est faux alors sa négation est vraie
 - si un prédicat est vrai alors sa négation est fausse

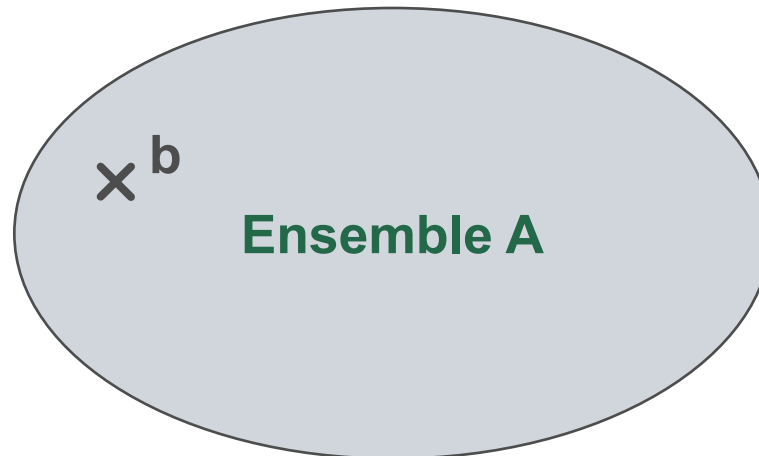




Rappels sur les prédicats

Le complément et la négation

- A l'instar des autres opérateurs, il existe une « équivalence » entre le négation et le complément.
- $(b \in A)$ est vrai donc :
 - $\neg(b \in A)$ est faux
 - $(b \in \bar{A})$ est faux

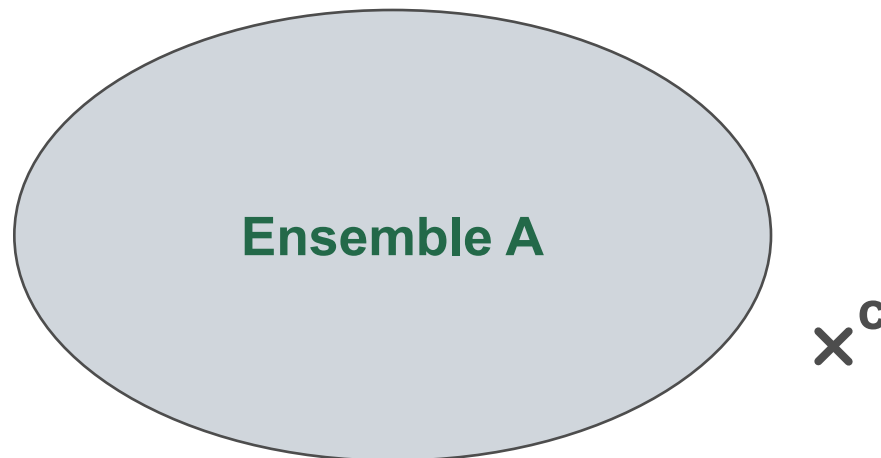




Rappels sur les prédicats

Le complément et la négation

- A l'instar des autres opérateurs, il existe une « équivalence » entre le négation et le complément.
- $(c \in A)$ est faux donc :
 - $\neg(c \in A)$ est vrai
 - $(c \in \bar{A})$ est vrai





Synthèse des 4 opérateurs

La table de vérité

- Les opérateurs qui ont été présentés précédemment, peuvent être décrits de manière plus concise grâce à une **table de vérité**. Il s'agit d'un tableau séparé en deux parties :
 - la partie de gauche indique l'état (vrai ou faux) des prédicats de départ
 - la partie de droite indique l'état du prédicat correspondant à l'opération





Synthèse des 4 opérateurs

La table de vérité

A	B	$A \vee B$
faux	faux	faux
faux	vrai	vrai
vrai	faux	vrai
vrai	vrai	vrai

A	B	$A \wedge B$
faux	faux	faux
faux	vrai	faux
vrai	faux	faux
vrai	vrai	vrai

A	B	$A \text{ ou }_e B$
faux	faux	faux
faux	vrai	vrai
vrai	faux	vrai
vrai	vrai	faux

A	$\neg A$
faux	vrai
vrai	faux





Synthèse des 4 opérateurs

Opérateurs unaires, binaires et ternaires

- Nous constatons que les opérateurs $\wedge$, $\vee$ et ou_e utilisent deux opérandes : ces opérateurs sont donc qualifiés de **binaires**.
- A l'inverse, l'opérateur $\neg$ n'utilise qu'une opérande, il est donc qualifié d'**unaire**.
- Nous verrons plus tard d'autres opérateurs qui peuvent prendre trois opérandes, ces derniers seront alors qualifiés d'opérateur **ternaires**.





George Boole

Présentation

- Le mathématicien **George Boole** a étudié l'algèbre dans un ensemble formé de deux éléments 0 (l'élément nul) et 1 (l'élément unité).
- Il a muni cet ensemble de 2 opérations de composition interne (notées **+** et **×**) et de 1 opération de complémentation (notée **−**).





Caractéristiques de $+$, $\times$ et $-$

L'opérateur $+$

■ Associativité :

$$\forall a, \forall b, \forall c, (a + b) + c = a + (b + c)$$

■ Commutativité :

$$\forall a, \forall b, a + b = b + a$$

■ Élément neutre :

$$\forall a, a + 0 = 0 + a = a$$





Caractéristiques de $+$, $\times$ et $-$

L'opérateur $+$

- Idempotence :

$$\forall a, a + a = a$$

- Élément absorbant :

$$\forall a, a + 1 = 1$$





Caractéristiques de $+$, $\times$ et $-$

L'opérateur $\times$

- L'opérateur $\times$ peut également être noté $\bullet$.

- Associativité :

$$\forall a, \forall b, \forall c, (a \bullet b) \bullet c = a \bullet (b \bullet c)$$

- Commutativité :

$$\forall a, \forall b, a \bullet b = b \bullet a$$

- Élément neutre :

$$\forall a, a \bullet 1 = 1 \bullet a = a$$





Caractéristiques de $+$, $\times$ et $^{\neg}$

L'opérateur $\times$

- Idempotence :

$$\forall a, a \bullet a = a$$

- Élément absorbant :

$$\forall a, a \bullet 0 = 0$$





Caractéristiques de $+$, $\times$ et $^{\sim}$

La distributivité

- La somme est distributive sur le produit :

$$\forall a, \forall b, \forall c, a \bullet (b + c) = (a \bullet b) + (a \bullet c)$$

- Le produit est distributive sur la somme :

$$\forall a, \forall b, \forall c, a + (b \bullet c) = (a + b) \bullet (a + c)$$





Caractéristiques de $+$, $\times$ et $\bar{}$

L'opérateur $\bar{}$

- L'opérateur $\bar{}$ peut également être noté $/$ (ou aussi $!$).
- La somme d'un nombre et de son complément donne 1 :

$$\forall a, a + /a = 1$$

- Le produit d'un nombre et de son complément donne 0 :

$$\forall a, a \bullet /a = 0$$





Caractéristiques de $+$, $\times$ et $\bar{}$

L'opérateur $\bar{}$

- Si deux nombre a et b sont complémentaires pour les opérations, $+$ et $\bullet$, alors ils sont complémentaires ($b = \bar{a}$).
- Par voie de conséquence, on a :

$$0 = \bar{1} \text{ et } 1 = \bar{0}$$

- Le complément du complément d'un nombre a est a :

$$\forall a, \bar{\bar{a}} = a$$





Principe de dualité et lois de (De) Morgan

Principe de dualité

- Une relation booléenne valide peut être obtenue à partir d'une autre relation booléenne valide en remplaçant :
 - les nombres par leur complémentaire :
 - 1 par 0 ;
 - 0 par 1;
 - a par $\neg a$;
 - $+$ par $\times$ (ou $\bullet$) ;
 - $\times$ (ou $\bullet$) par $+$.





Principe de dualité et lois de (De) Morgan

Loi de De Morgan

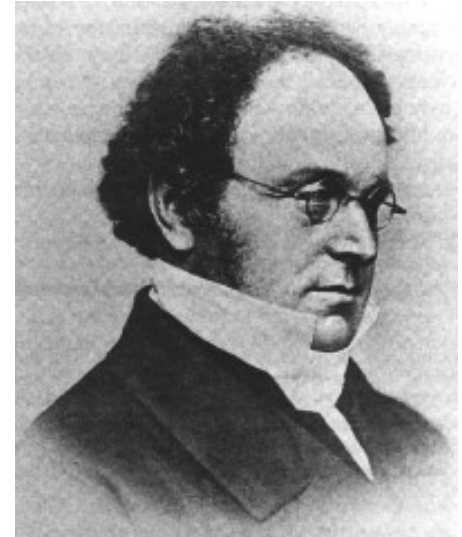
- Le mathématicien **Auguste De Morgan** a montré les deux lois suivantes :

- Le complément d'une somme est égal au produit des compléments :

$$\forall a, \forall b, /(a + b) = /a \bullet /b$$

- Le complément d'un produit est égal à la somme des compléments :

$$\forall a, \forall b, /(a \bullet b) = /a + /b$$





Analogie entre $\vee$, $\wedge$, $\neg$, et $+$, $\times$, $\bar{}$

Le principe

- A l'instar des opérateurs $\wedge$, $\vee$ et $\neg$, les opérateurs $+$, $\times$, $\bar{}$ peuvent être présentés sous forme d'une table de vérité.
- Si nous associons 1 à vrai et 0 à faux, nous constatons qu'il existe une équivalence entre
 - l'opérateur $+$ et l'opérateur $\vee$
 - l'opérateur $\times$ et l'opérateur $\wedge$
 - l'opérateur $\bar{}$ et l'opérateur $\neg$





Analogie entre $\vee$, $\wedge$, $\neg$, et $+$, $\times$, $-$

Les opérateurs $\vee$ et $+$

A	B	$A \vee B$
faux	faux	faux
faux	vrai	vrai
vrai	faux	vrai
vrai	vrai	vrai

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1





Analogie entre $\vee$, $\wedge$, $\neg$, et $+$, $\times$, $-$

Les opérateurs $\wedge$ et $\times$

A	B	$A \wedge B$
faux	faux	faux
faux	vrai	faux
vrai	faux	faux
vrai	vrai	vrai

A	B	$A \times B$
0	0	0
0	1	0
1	0	0
1	1	1





Analogie entre $\vee$, $\wedge$, $\neg$, et $+$, $\times$, $^-$

Les opérateurs $\neg$ et $^-$

A	$A \neg B$
faux	vrai
vrai	faux

A	$/A$
0	1
1	0





L'opérateur $\oplus$

La construction de l'opérateur et sa table de vérité

- Nous ajoutons un nouvel opérateur qui est défini de la manière suivante :

$$\forall a, \forall b, a \oplus b = (a \bullet /b) + (b \bullet /a)$$

- Cet opérateur correspond « naturellement » à l'opérateur ou_e décrit précédemment.

A	B	A ou_e B
faux	faux	faux
faux	vrai	vrai
vrai	faux	vrai
vrai	vrai	faux

A	B	/A	/B	A $\bullet$ /B	B $\bullet$ /A	A + B
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0





Les tables de Karnaugh

Le principe

- La **table de Karnaugh** est un outil qui a été inventé par **Maurice Karnaugh**, un ingénieur télécom des Bells Labs.
- Elle permet de simplifier des expressions booléennes qui se présentent sous la forme d'une suite de « ou logique » dont les membres sont des « et logique » entre des variables de base.
- Exemple :

$$F = (A \wedge (\neg B) \wedge C) \vee (A \wedge B \wedge (\neg C)) \vee ((\neg A) \wedge B \wedge C)$$





Les tables de Karnaugh

Le principe

- Le principe d'utilisation est le suivant :
 - construire la table de telle manière que l'on passe d'une variable à son complémentaire lorsqu'on change de ligne ou de colonne
 - placer dans le tableau, des croix qui correspondent aux variables apparaissant dans l'expression booléenne à simplifier
 - mettre en évidence les croix adjacentes qui indiquent la présence d'un « ou logique » entre une variable et son complémentaire dans une partie de l'expression
 - écrire l'expression intermédiaire correspondant au groupement des croix : l'expression finale est un « ou logique » entre ces expressions intermédiaires





Les tables de Karnaugh

Exemple avec 3 variables

- On considère l'expression booléenne suivante :

$$F = (A \bullet (!B) \bullet C) + (A \bullet B \bullet (!C)) + ((!A) \bullet B \bullet C)$$

- On construit d'abord une table de Karnaugh vide
- On place alors les croix correspondant à l'expression.

	AB	A(!B)	(!A)(!B)	(!A)B
C		X		X
!C	X			

- On constate alors que F n'est pas simplifiable car on ne peut pas grouper les croix.





Les tables de Karnaugh

Un autre exemple avec 3 variables

- On considère cette fois l'expression booléenne suivante :

$$F = ((\textcolor{red}{!A}) \bullet \textcolor{red}{B} \bullet (\textcolor{red}{!C})) + (\textcolor{teal}{A} \bullet \textcolor{teal}{B} \bullet (\textcolor{teal}{!C})) + ((\textcolor{blue}{!A}) \bullet \textcolor{blue}{B} \bullet \textcolor{blue}{C}).$$

	AB	A(!B)	(!A)(!B)	(!A)B
C				X
!C	X			X

- Le groupe formé par les croix bleue et rouge correspond à l'expression $(!A)B$ et le groupe formé par les croix verte et rouge (le tableau étant circulaire) correspond à l'expression $B(!C)$.
- F est donc finalement égal à $(!A)B + B(!C)$.





Les tables de Karnaugh

Un autre exemple avec 3 variables – Vérification

- On construit la table de vérité afin de comparer les deux expressions : on constate que les deux colonnes sont identiques donc les expressions sont équivalentes.

A	B	C	$((!A) \bullet B \bullet (!C)) + (A \bullet B \bullet (!C)) + ((!A) \bullet B \bullet C)$	$(!A)B + B(!C)$
0	0	0	0	0
0	0	1	0	0
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0





Les tables de Karnaugh

Exercice avec 4 variables

- Simplifier l'expression :

$$F = AB(!C)(!D) + (!A)B(!C)D + (!A)BC(!D) + (!A)B(!C)(!D) + (!A)BCD$$

	AB	A(!B)	(!A)(!B)	(!A)B
CD				
(!C)D				
(!C)(!D)				
C(!D)				





Les tables de Karnaugh

Exercice avec 4 variables

- Simplifier l'expression :

$$F = AB(!C)(!D) + (!A)B(!C)D + (!A)BC(!D) + (!A)B(!C)(!D) + (!A)BCD$$

	AB	A(!B)	(!A)(!B)	(!A)B
CD				X
(!C)D				X
(!C)(!D)	X			X
C(!D)				X

- En groupant X, X, X et X, on obtient $(!A)B$ et en groupant X et X, on obtient $B(!C)(!D)$
- Finalement, on a $F = (!A)B + B(!C)(!D)$





Les tables de Karnaugh

Passage table de vérité vers table de Karnaugh

- On peut construire une table de vérité depuis une table de Karnaugh. Pour cela, il faut :
 - énumérer les différentes combinaisons entre les variables et leurs complémentaires dans la partie gauche
 - placer, dans la partie droite du tableau, des 1 correspondant au X du tableau (et des 0 pour signifier l'absence de X)





Les tables de Karnaugh

Passage table de vérité vers table de Karnaugh – Exemple

	AB	A(!B)	(!A)(!B)	(!A)B
CD				
C(!D)				
(!C)D			X	X
(!C)(!D)			X	X

A	B	C	D	Expression
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0





Pause-réflexion sur cette 4^{ème} partie

Avez-vous des questions ?





Résumé du module

Algèbre de Boole : aspect théorique de l'électronique numérique

Analogique versus Numérique

Codage des caractères : ASCII et UNICODE

Codage des entiers et arithmétique binaire

Codage des réels : norme IEEE 754

